

IBM i Coding Standards

Baseline standards for RPGLE/SQLRPGLE, CL/CLLE, SQL, CMD, PNLGRP, and DDS source in **all Godsend Consulting IBM i repositories** — GCUTL, BSLIB, RBUTL, and any future project. They were derived from the current best practice in these libraries — primarily the Task Manager (TM*) and developer-utility (RB*) members in **RBUTL**, which is the curated exemplar. BSLIB additionally carries legacy, third-party, and scratch code that does *not* set the standard (see section 11, Legacy and scratch code).

Canonical copy: the master of this document lives in the **IBMI-STANDARDS** repository; every IBM i repo carries an identical copy so the standard travels with the code. Edit the master, then run its **sync-standards.ps1** to refresh the repo copies — don't edit a repo copy directly.

Precedence: when this document and an existing member disagree, follow this document for new code. When this document is silent, match the closest RBUTL sibling of the same member type.

The IBM manuals are on this machine. Read them before reasoning from first principles. A local, maintained set lives in:

C:\Users\Public\Documents\Programming\IBM i Manuals

Around eighty PDFs, covering nearly everything this document touches — *ILE RPG Language Reference* and *Programmer's Guide*, *CL Overview and Concepts*, the three DDS manuals, *SQL Reference*, *SQL Programming*, *Embedded SQL Programming*, *API Overview and Concepts*, *ILE Concepts*, *Application Display Programming* (the UIM tag reference behind section 7), the REXX pair, and a shelf of Redbooks.

Some of it IBM no longer publishes, so treat the directory as irreplaceable rather than merely convenient. The ADTS set is the clearest case — *Source Entry Utility*, *Programming Development Manager*, *Screen Design Aid*, *Report Layout Utility*, *Data File Utility* — documenting tools this shop uses daily and for which there is no current edition to download. The RPG/400 pair is there for the same reason: fixed-form OPM code still exists in these libraries (section 11) and nothing current describes it. Do not assume a manual can be re-fetched; back the directory up as source, not as cache.

A PDF that size does not read, it greps. Extract once and search the text:

```
pdftotext -layout "IBM i Manuals\Application Display Programming [sc415715].
```

Several rules in sections 6 and 7 were worked out the expensive way and then found stated plainly in that manual. One was corrected only after the manual was finally consulted — the standard had recorded a formatting *default* as though it were a constraint. **Check the manual first; it is faster than the experiment and it is right.**

1. Universal member conventions

These apply to every source member regardless of language.

1.1 Standard header

Every member begins with, in order:

1. **Purpose comment** — what the member does, plus any design notes worth a paragraph. Write *why*, not just *what*.
2. **MODIFICATIONS: log** (see 1.2).
3. **BLDOBJ build directives** (see 1.3).
4. **Copyright** — (c) Copyright Godsend Consulting, <first year>—<current year>. In RPG this is the `ctl-opt copyright( ... )` keyword; in CL the `COPYRIGHT` command; elsewhere a comment line (CMD) or `:COPYR.` tag (PNLGRP).

Comment syntax per language:

Language	Block comment	Build directive line
RPGLE (**FREE)	<code>/// doc block / //</code>	<code>// *> <BLDOBJ CRTDFT /></code>
CL, CMD	<code>/* ... */</code> with floating + continuation	<code>/*> CRTCMD ... <*/</code>

In CL and CMD, EVERY line of a multi-line comment ends with +. The continuation is not decoration on the **MODIFICATIONS** entries — it is what makes the comment one statement. Without it the compiler takes each line as its own, and the ones that do not open with `/*` are not comments at all:

```
/* Copies a job schedule entry to a new job name. +
+
  ENTRYNBR selects which entry when the job name is not unique - +
    the same job name may hold several entries. +
*/
```

A blank separator line inside the block is a bare `+`, as above. **A hyphen at the end of a line is lost**, because the `+` takes that position — so end a line on a word and put the dash mid-line, or the sentence reads as a run-on after the next edit. | SQL | `/* ... */`,
-- inline | `/*> RUNSQLSTM ... <*/` | | PNLGRP | `.* | .*> <BLDOBJ CRTDFT />` | |
DDS (DSPF) | `_* doc block / A* | *> CRTDSPF FILE(&O/&N) ... -` |

1.2 MODIFICATIONS log

Every change appends a dated entry. Format is fixed — tools and habit depend on it:

MODIFICATIONS:

BS 08/03/25 Created.

BS 06/15/26 Updated documentation to acknowledge that default values for an undefined location effectively remove all restrictions that can be set for a location.

@@

- XX MM/DD/YY *description* — contributor initials, US date, sentence-case description ending in a period.
- Entries are chronological, oldest first; **append new entries at the bottom**, immediately before the @@ terminator line.
- Continuation lines align under the description, indented one extra space.
- **A change spanning more than one day carries a date RANGE:** the start date in the date column of the first line, the end date in the same columns on the **second** line, with the initials column left blank. Only two dates are ever shown, and only when the work actually crossed a day — a same-day change has no second date. Where the description needs no second line, the end date stands alone on one:

MW 09/07/18 Do not purge POSCL if the filegroup or blackout is not populated for some reason.

MW 06/21/19 Do not enforce Louise Paris edit.

BS 11/22/19 Add question to allow blackout to be
12/22/19 split by style if the buyer has been
configured to allow this feature.

MW 06/21/20 Some change 2-day change text here.
06/22/20

MW 07/21/20 The last change

@@

- One entry may describe several changes made together; start each on its own continuation line.
- The @@ line terminates the log. Never remove it.
- @todo lines may follow the log (before the build directives) to record known future work. **One @todo per item** — two things to do are two lines, not one line with two sentences, so that each can be read, counted and struck off on its own. **Lowercase**, like every other ILEDoc command (§2.3): @todo, never @TODO or @ToDo.

1.3 BLDOBJ build directives

There is no external build system. Each member embeds its own create/compile command(s) between *> and <*> markers; the BLDOBJ command extracts and runs them on the IBM i (see QRPGLSRC/BLDOBJ .SQLRPGLE header for the full reference).

- If the default create command for the member type is correct, use:

```
/*> <BLDOBJ CRTDFT />
/*> <BLDOBJ EOF />
```

```
<*/
<*/
```

- Otherwise spell out the command(s) with substitution variables — &O object library, &L source library, &F source file, &N member, &ON object name, &T type, &X member text:

```
/*> CRTCMD CMD(&O/&ON) SRCFILE(&L/&F) SRCMBR(&N) PGM(TMACRUSR)
/*> <BLDOBJ EOF />
```

```
<*/
<*/
```

- Prefix a command with **IGN:** when its failure must not stop the build (e.g. **IGN:DLTF QTEMP/OUTPUT**).
- Always end with **<BLDOBJ EOF />** so scanning stops. **Without it BLDOBJ reads the whole member** looking for directives it will not find, on every build. That is the entire job of the tag — it bounds the scan and nothing else.
- **CRTDFT and EOF are not the same kind of tag.** CRTDFT inserts the default create command for the member type; EOF stops the scan. A member with neither still builds, because BLDOBJ derives the same default once it has read to the end and found nothing — so CRTDFT buys clarity and EOF buys the read.
- Keep directives correct when copying a member — that is its build script.

Declare in the source whatever the source can declare, and the directive stays CRTDFT. Most of what looks like it needs a spelled-out create command does not, because the same attribute has a source form:

Attribute	Source form	Not
help panel group, help id	HLPID / HLPPNLGRP on the CMD statement	CRTCMD parameters
validity checker	VLCKR on the CMD statement	a CRTCMD parameter
choice program	CHOICEPGM on the PARM	anything at create time
bound service program	BNDSRVPGM on DCLPRCOPT (CL)	a spelled-out CRTBNCL
binding directory	BNDDIR on DCLPRCOPT or ctl-opt	a spelled-out create
activation group	ACTGRP on DCLPRCOPT	a create parameter

A custom command is forced only by something with **no** source equivalent — **PGM()** on a command whose CPP is named differently, or the CRTPF keywords a DDS member cannot carry. Everything else belongs in the member, where a reader finds it.

The failure mode is silent. An attribute that lived only on a hand-written build script disappears when the script is replaced by CRTDFT, and the object is created without it. A command whose HLPPNLGRP was on its old CRTCMD builds perfectly and has no help.

Member text lives in the source. Source edited off-platform round-trips through the IFS, and a source physical file member's **text description does not survive that trip**. The only place it can be kept is the member itself, so every member carries its own:

Member type	Where the text goes
.CMD	TEXT('...') on the CMD statement
CL (.CLLE, .CLP)	TEXT('...') on DCLPCROPT
Everything else	*> <BLDOBJ TEXT="..." />

So altering the source is what preserves the source. This applies even to members that are never built — a reference-only layout still has member text worth keeping, and a TEXT directive carries no create command, so it does not make the member buildable.

Put it immediately after the @@. Two reasons, and the second is the general one:

- @@ is the marker everyone navigates to, because appending a **MODIFICATIONS** entry means finding it and working back. A directive on the line after it is seen by anyone who touches the member; the same directive above the include guard, or anywhere else in the header, is documentation nobody scrolls to.
- It is **one location for every member type**. A buildable member already carries its create commands there, so putting the text directive in the same place means source-only and source-to-object members read alike — you look after the @@ and find whatever that member has, without first working out which kind it is.

Text at the top is better than no text at all. A member carrying the directive somewhere else in the header is not broken and is not worth a sweep; move it down when you are in the member for another reason.

In RPG, the directives live INSIDE the ILEDoc block — after the @@, with the closing /// after them:

```
///
// Purpose of the member ...
//
// MODIFICATIONS:
// BS 08/20/26 Created.
// @@
// *> <BLDOBJ CRTDFT />
// *> <BLDOBJ EOF />
///
```

This follows from the rule above rather than adding to it: the point of putting the directive after the @@ is that whoever appends a **MODIFICATIONS** entry sees it, and a closing **///** between the two separates them for no reason. The header is one block, so it closes once, at the end.

It applies to **every** RPG member — program, module and copybook alike. There is nothing about a copybook that makes its header different from a program's.

CL is left alone, for now. CLLE supports ILEDoc too, but nothing in these repos uses it — not one **@param** across **QCLSRC**, against several hundred in **QRPGLESRC** — and it is not established that anything reads it, including the VS Code extension. So a CL member keeps its **/*> ... <*/** directive lines as they are: there is no block for them to be inside of, and inventing one would mean choosing a convention with no consumer to validate it. Not a gap to be filled by whoever notices it next.

1.4 Layout

- **Line width.** The hard maximum is the source file's **SRCDTA length** — the source line itself — and a longer line is **truncated silently**:

Source file	Line (SRCDTA)	Record length
QRPGLESRC	100	112
QPNLSRC	134	146
QTXTSRC	132	144
Everything else — QCLSRC, QCMDSRC, QSQLSRC, QDDSSRC, QREXSRC, QMNUSRC, ...	80	92

Measure against the SRCDTA column, never the record length. A source physical file record carries a 12-byte prefix — a 6-byte sequence number and a 6-byte date — that is not part of your line. Quoting the record length as a line limit hands you 12 characters that do not exist, and the overflow is discovered by truncation. That mistake is exactly how the **XFENV . SQLT** lines below got through review at 81 and 83 characters in an 80-column file.

RBUTL practice is ~76, and these are a limit rather than a budget to spend.

Nothing warns you when you exceed it. The line is cut when the member is written or transferred, and the loss surfaces later as a compile failure — or worse, as code that still looks right with a character missing. The dangerous case is a truncated string literal: lose the closing quote and the parser runs on into whatever follows, so the error is reported nowhere near the damage. This has actually happened here (**XFENV . SQLT**, two **LABEL ON COLUMN** lines at 81 and 83

characters in an 80-wide file, both cut to exactly 80). Any tool that rewrites source in bulk will do this to every over-length line at once.

An over-length line is not the only cause of truncation, and the symptom is identical. A build or source-manipulation tool that stages members through a work file — typically QTEMP/QSQLSRC or QTEMP/QSQLTEMP — truncates to *that* file's record length, whatever your lines measure. If the work file was created earlier by something else, at the wrong width, it persists for the life of the job and silently cuts every member that passes through it.

Diagnose in this order:

1. **Measure the longest line in the member.** If nothing exceeds the limit in §1.4, the member is not the problem and no amount of reformatting will fix it. Verified XFBLD . SQLT, whose longest line was 79 in an 80-column file, while the build still failed.
2. **Delete the QTEMP work files and let the system recreate them** — DLTF QTEMP/QSQLSRC, and QTEMP/QSQLTEMP with it. Recreated on demand, they come back at the correct width.

The failure surfaces from wherever the truncated text lands, which is rarely where the truncation happened. A compound **CREATE OR REPLACE** built as dynamic SQL goes to the SQL precompiler, which generates C and compiles it — so a cut line is reported as a **C compile error**, naming neither your member nor the work file. Balanced quotes plus a C-level error is the signature: the source is fine and something between it and the compiler is not.

- **Hybrid RPG** (no ****FREE**): fixed-form specs are, well, fixed — their columns aren't a style choice. Free-form lines in a hybrid member must fit **columns 8–80** whatever the record length, because the compiler does not read free-form code past column 80.
- **PNLGRP**: a **:HELP** title that would exceed 80 stays on one line (see §6) rather than wrapping or being abbreviated — that is what the 134 is for.
- **Section separators**: a full-width comment rule between major sections and a shorter/dashed rule between minor groups:
 - RPG: `// ***** ... *****` (major), `// ——— ... ———` (minor)
 - CL/SQL: `/* ***** ... ***** */`
 - PNLGRP: `.* ***** ... *****` with a `.* Title + .* ———` underline per section
- **Blocks read top-down**: mainline first, error handling next, subroutines/procedures last, each introduced by a one-line comment saying what it does.

1.5 Naming and member families

- **Prefixes:** TM* / *TM = Task Manager, RB* = developer utilities, BS* = BSLIB personal utilities. JUNK* = scratch (never promote, git-ignored).
- A user-facing command is a **family sharing one base name**:
 - QCMDSRC/NAME.CMD — command definition (HLPPNLGRP(NAME), MSGF(TMMMSG))
 - QCLSRC/NAME.CLLE or an RPGLE — command-processing program (CPP)
 - QPNLSRC/NAME.PNLGRP — command help
 - messages come from the shared TMMMSG message file (QREXSRC/TMMMSG.REXX)
- **RPG member suffixes:** base name = program/entry, ...H = prototypes/constants copybook, ...P = service-procedure implementations. Example: RTVLOCTM + RTVLOCTMH. An ...R suffix (subprocedure-only module, e.g. SQL2XLSXR) appears in third-party code and tool-generated members — recognize it as a pattern, but it is not required for new members.
- Related CRUD commands funnel into one shared CPP distinguished by a `CONSTANT(*ADD|*CHANGE|...) MODE` parameter (e.g. TMACRUSR, TMCCDTSK).
- **Symbol namespaces — use the full object name, never a truncation.** A member that publishes names into other members — any ...H copybook, and the ...P implementation copybook that goes with it — prefixes **every** constant, template, and procedure it publishes with its own full name.

The namespace exists to prevent collisions, and that is a requirement, not a preference. Every name a member publishes lands in the symbol table of every program that pulls it in, alongside the names from every other member pulled into the same program. Without a namespace those names compete, and the failure is a compile error at best and the wrong procedure at worst. That a prefix also tells a reader where a name came from is a large bonus — but it is the bonus, not the reason. Do not weigh the bonus against the cost and conclude a namespace is optional; the requirement is not negotiable.

Kind	Form	Example
Constant	@FULLNAME_SCREAMING_SNAKE	@GUXLINF_ATR_MAX
Template	fullname_lowercase_t	guxlinf_infatr_t
Procedure	fullname_camelCase	guxlinf_getSheetName

The namespace is the object name **in full**. An older IP convention dropped the two-character application id and namespaced on the remainder — GUXLINF publishing `xlinf_*` — and that is no longer used anywhere. Do not reintroduce it,

and never mix the two within one member: @GUXLINF_ATR_MAX sitting beside `xlinf_infatr_t` is the specific defect this rule exists to prevent. The full name is what a reader greps for and what WRKOBJ shows; a truncation is a second name for the same thing.

This applies to a /copy copybook exactly as it does to a bound service program — a copy member's names carry the same collision exposure as a *SRVPGM's exports, which is why GUSQCPYP namespaces its procedures despite being copied rather than bound.

A copybook shared by only one closed family of programs is not exempt. The argument for exempting one is that nothing outside the family will ever pull it in, so nothing can collide — but that premise is a claim about the future, and the include graph is what actually decides it. Copy members include each other, sometimes mutually behind their guards, so a member three includes away can acquire the whole set without anyone intending it. Check before believing the premise, and expect it to have already failed. TMXXXWRKH is the worked example: it serves the WRK*TM panel family, yet RTVLOCTM — a retrieve CPP, not a panel program — receives every one of its names through RTVLOCTMH, which TMXXXWRKH in turn includes.

Two things keep their own names rather than taking the copybook's:

- **External-file templates**, which are namespaced by the table they mirror — `dcl-ds tmusr_t extname('TMUSR')`. Two copybooks may declare the same one behind an /IF NOT DEFINED guard so both can be pulled in.
- **Names owned by IBM or a third party** — the C types that arrive with /copy QSYSINC/QRPGLESRC, IFS (size_t, mode_t, pid_t), IBM API structure names, and third-party members, which keep their original spelling so they still match the documentation.

Constants are namespaced by **what they describe**, which is not always the member that declares them. Three cases, all correct:

- **Owned by the publishing member** — @GUXLINF_ATR_MAX. Takes the member's full name, as above.
- **Owned by another object** — @WRKOBJTM_PNLID belongs to the WRKOBJTM panel group (see section 6), @QUIM_* to UIM. Takes that object's name, which is what keeps the panel-group constants matching their :VARRCD and :LISTDEF names.
- **Owned by a concept rather than an object** — @OPTNBR_* for list option numbers, @QUALOBJ_* for qualified object names. There is no object to name, so a stable category prefix serves.

The third case is a **fallback that has to be justified, not a free choice**. Use it only when no object owns the concept; if one does, its name wins. The risk being accepted is that nothing stops a second member declaring its own @OPTNBR_*,

and two copy members pulled into one program would compete. That surfaces as a duplicate-definition compile error where the collision happens rather than as wrong behaviour at run time, which is what makes it tolerable — but only while the set of programs that can see the constant stays small.

@SQLCODE_* was in this list and has been taken out of it, which is the case working as intended. The SQLCODE values had no owner, so three separate members each declared their own — and the moment two of them could be pulled into one program, the category prefix was a collision waiting to happen. Giving them an owner resolved it: GUSQCPYH already published @GUSQCPY_SQLSTATE_CLASS_*, so SQLCODE belonged beside it as @GUSQCPY_SQLCODE_*. **When a concept turns out to have a natural owner, move it — the fallback is not a resting place.**

So category prefixes must not appear in a general-use copybook. A member written to be pulled in broadly — the API wrapper copybooks (QAPIH, QILEH, QUIMH), a library-wide utility header, anything a new program is expected to / copy as a matter of course — declares **only** constants in its own namespace. Two reasons: its reach is exactly what makes a category name likely to collide, and every name it declares is imposed on every consumer whether that consumer wants it or not. A category prefix is only defensible where the consumer set is bounded and known, as in a copybook serving one application family. Test it by asking who can see the constant: if the answer is "anything that might be written later," take the member's namespace instead.

So the rule is that a namespace is the full name of whatever owns the symbol — not that every symbol in a file carries that file's name.

Local subprocedures inside a single program are not published and take the plain <verb><Object> form of section 2.5.

- **Object-role suffixes.** Object names follow a 2+2+3 shape — application, area, object. Where the third element has no entity to describe, because the object is infrastructure rather than application, it names the object's **role** instead. These are reserved across all repos:

Suffix	Object type	Role
...SRV	*SRVPGM	Bound service program — the API other programs call
...AGT	*PGM	Agent — a never-ending job that processes queued work
...REQ	*FILE	The request table an agent drains
...DTQ	*DTAQ	Data queue, where one is used as a doorbell

SVR is **never** used for a service program. SRV and SVR sort adjacent in WRKOBJ, are one transposition apart when typing, and are indistinguishable when spoken aloud. Pick SRV and never write the other.

Prefer a role suffix that stays true for the whole life of a row or object. ...REQ beats ...QUE for the request table because a sent row is still a request that was fulfilled, whereas it is no longer queued — and because QUE reads as *DTAQ/*MSGQ in an object listing.

1.6 Agents — the never-ending job pattern

An **agent** (...AGT) is a submitted job that runs indefinitely: it wakes on a timer or a queue, does one unit of work, and goes back to sleep. This is what an AS/400 shop has always called a *server job* or a *NEP* (never-ending program) — the pattern is old, only the name is new.

AGT is the right name for it because the term already belongs to this platform: STRAGTSRV / ENDAGTSRV and the CMDAGT menu are IBM's own Electronic Service Agent. "Service Agent" there is a compound proper noun for an IBM facility, not a composition of the SRV and AGT suffixes, so there is no conceptual collision with a ...SRV service program. The alternatives lose: MON implies passive watching or dispatch only, when the job actually acts; DMN is Unix vocabulary and misreads as "domain".

A subsystem built this way is a **family of four**, named together:

Member	Type	Responsibility
xxSRV	*SRVPGM	Validates and enqueues. What applications bind to.
xxREQ	*FILE	The request rows. System of record.
xxAGT	*PGM	Drains ready rows, dispatches, housekeeping, breaker.
xx<verb>	*PGM	Performs one unit of work, records its disposition.

Rules that keep the family honest:

- **The agent and the worker are separate programs — never one program with a mode parameter.** A bare CALL on a merged program starts a second never-ending job draining the same queue, which is lock contention at best and duplicate work at worst. The job log stops distinguishing the all-day daemon from a single unit of work. And the two have genuinely different lifecycles: the agent holds file opens and state across thousands of iterations, while the worker opens, works, and ends. Merging them means conditional initialization inside a NEP, which is where the subtle bugs live.

- **The worker writes the row's final status; the agent never does.** The agent reads return codes only for its own arithmetic. Two writers to one status column works fine until a unit of work takes longer than expected.
- **The agent claims the row before dispatch; the worker takes a lock.** The agent moves the row out of the ready status and stamps a *claim timestamp* before it calls or submits the worker. It has to: SBMJOB returns as soon as the job is queued, so a row left in ready status is re-selected and submitted again on the agent's very next pass.

The worker then **fetches the row by key and expected status, with a lock, and holds that lock for the whole unit of work.** Put the status in the predicate rather than testing it after the read — a row in the wrong status is simply not found, so there is no lock to release and no branch to get wrong. Not found is a silent no-op: no error, no escape, just a diagnostic in the job log. The lock — not the predicate by itself — is what serializes two workers, keeps a human from re-flagging mid-flight, and lets an accidental double dispatch resolve itself.

- Name the claimed status for what is true of it. A submitted row can sit on a job queue for a long time, so *queued* is honest where *running* or *sending* would not be.
- **The worker stamps its own qualified job name** when it accepts the row — not the agent, which would have to dig the job id out of SBMJOB's completion message. Leaving the column empty until a worker actually starts is the more useful design anyway: it lets the sweep tell "never started" (dead job queue, held subsystem) from "started and died," which recover differently.
- **The worker registers a termination handler**, sets a flag while it holds a locked row, and has the handler record the failed status and release. An ILE cancel handler (CEERTX) or RPG ON-EXIT covers unhandled exceptions, function checks, and ENDJOB/ENDSBS *CNTRLD. This is the primary recovery path: the outcome is recorded by the job that knows what happened, seconds after it happens, instead of being inferred later by another program.
 - It does **not** run for ENDJOB *IMMED, system failure, or abnormal IPL.
 - The handler usually cannot know whether an external side effect already succeeded, so word its disposition as *the job ended abnormally* — not *the work was not done*. Only the first is true.
- **The sweep is a backstop, not the recovery path.** With a termination handler in place it exists for the cases where no program was alive to record anything: a worker that never started, and one killed outright. It reads without locks — reasoning about rows, not changing them — and thresholds on the claim timestamp, which covers the legitimate gap between claim and pickup.
- **Recover by returning the row to the ready status, not by re-dispatching it.** The agent picks it up again through the normal path, so the sweep never duplicates dispatch logic; the trigger clears the claim fields on the way through; and it throttles itself, because a recovered row is immediately re-

claimed with a fresh timestamp and cannot re-qualify until the threshold passes again. Re-dispatching in place leaves the original timestamp untouched and re-qualifies the same rows every tick, each one adding another worker to the queue it is already waiting on.

- **Recovery is only free where nothing was done yet.** The lock serializes concurrent workers; it says nothing about a sequential one. A worker that completed an external side effect and died before recording it leaves a row that looks identical to one never started — recovering it does the work twice. Use the worker-stamped job name to tell them apart: absent means nothing started; present means the outcome is unknowable and the decision belongs to whatever low-stakes/high-stakes split the subsystem already has.
- **Every side effect of a status change belongs in the table's trigger, not in the programs.** Programs set the status; the trigger does the rest, keyed on a status *change* (compare OLD to NEW, and let an unchanged status fall through so the worker can stamp its own job name). Statuses that re-arm a row clear the outcome fields — a retried request must not carry the previous attempt's error text — while statuses that record an outcome leave them alone. Done in the programs instead, the fields are only as honest as the last one to touch the row, and say nothing about the green-screen path where someone re-runs work by typing over the status. In the trigger, no path can forget, including paths that do not exist yet.
- **A processed timestamp, set by the trigger only on the terminal statuses,** and cleared when a row is re-armed. Null for in-flight rows and non-null for finished ones, it makes "not yet processed" a property of the data rather than a list of status codes each query has to know, and the retention pass selects on it without status logic.
- **Time in-flight rows by audit_timestamp,** not by their created timestamp. A row's age since *creation* says nothing about how long it has been claimed: after an agent outage, every row it then claims would look instantly stale and a recovery sweep would start fighting the agent that just claimed them. The mandated **audit_timestamp** moves on any update, so for a claimed row it means "nothing has touched this since," which is the actual question — and it costs no extra column.
- **Age unprocessed rows separately from processed ones, and never delete them silently.** Retention on finished rows is routine; a row that grows old while still in flight is evidence of a bug or an outage, and deleting it destroys the only record that the work never happened. Move it to the failed status with a disposition saying it expired unprocessed, alert out of band, and let it age out through the normal retention path.
- **Do not pre-load the worker's job name at claim time,** even where it is technically available (an inline path knows its own job; a submitted one can recover the job id from SBMJOB's CPC1221). The column earns its place by meaning "a worker picked this row up." Fill it at claim time and every row has a name from the moment it is claimed, the never-started and died-midway

cases become indistinguishable, and recovery is back to guessing. The empty column is carrying information.

- **The agent must release its own claim lock before dispatching.** On an inline path the worker runs in the *same job* and requests the same record through its own open, which is a lock conflict rather than a free pass. This works in isolation and fails when the two programs are first wired together, so prove it early.
 - **Decide what recovering an orphaned row means before writing the sweep.** There is always a window where the worker finished its unit of work but died before recording the outcome, so requeueing risks doing the work twice while failing the row risks not doing it at all. Choose per the cost of each — and where the subsystem already distinguishes low-stakes from high-stakes work, let that existing split decide rather than inventing a second one.
 - **Shut down by flag, not by cancel.** A run/stop indicator in a data area or control record, tested at the top of each pass, ends the job cleanly between units of work rather than mid-unit.
 - **Whatever the agent talks to** — a command, an API, an external transport — **appears in exactly one program**, so replacing it touches one object.
 - **Count failures for the circuit breaker where every path is visible.** An agent that both calls a worker inline and submits it for some rows sees return codes only on the inline path. Counting recent failures during the housekeeping pass catches both, and also catches submitted jobs that died without writing back.
-

2. RPGLE / SQLRPGLE

2.1 Form

- **All new RPG is fully free-form:** ****FREE** on line 1, column 1.
- Use **.SQLRPGLE** when the member contains embedded SQL, **.RPGLE** otherwise.
- Never write new fixed-form or hybrid code. Convert opportunistically when a fixed member needs nontrivial changes (CVTRPGFREE/CVTRPG1511 tools exist for this).
- When maintaining fixed-form **D specs**: subfields indent the **name only**, by one space — nothing else shifts. Keywords always start in the first column of the keyword area (column 44); don't stagger them per line:

D	PgmSts	SDs	Qualified
D	PgmNam	*Proc	
D	CurJob	10a	Overlay(PgmSts: 244)
D	UsrPrf	10a	Overlay(PgmSts: 254)

2.2 Header and control options

```
**FREE
///
//  Retrieves the attributes for the requested location.  If the
//      location has not been setup in the Task Manager, default
//      values will be returned.
//
//  MODIFICATIONS:
//  BS 08/03/25 Created.
//  @@
///
//  *> <BLDOBJ CRTDFT />
//  *> <BLDOBJ EOF />

ctl-opt copyright('(c) Copyright Godsend Consulting, 2020-2026');
ctl-opt option(*NODEBUGIO: *NOUNREF);
ctl-opt debug(*CONSTANTS);
ctl-opt decedit('0. ');
ctl-opt expropts(*ALWBLANKNUM: *USEDECEDIT);
ctl-opt extbinint(*YES);
/IF NOT DEFINED(*CRTRPGMOD)
ctl-opt dftactgrp(*NO);
ctl-opt actgrp(*CALLER);
/ENDIF
```

- One keyword per `ctl-opt` statement, **copyright** first.
- The standard option set is exactly the block above. Add `bnddir( ... )`, `alwnull(*USRCTL)`, etc. only when needed.
- Guard binding-only keywords with `/IF NOT DEFINED(*CRTRPGMOD)` so the member compiles as either a bound program or a module.
- Activation group: `*CALLER` for called/service-style programs; `*NEW` for top-level interactive programs (work-with panels, menus).

ctl-opt takes BNDDIR. It has never taken BNDSRVPGM. CL's DCLPRCOPT takes both, so a CL program names a service program it binds and keeps its build directive at CRTDFT (§1.3). **An RPG member cannot** — there is no source form, so an RPG program or service program that must name what it binds needs either a binding directory or a spelled-out create command.

Reach for the binding directory. A spelled-out create is a second place the build lives, and it is the place nobody looks.

Most of the time nothing needs naming at all. The Q* system service programs resolve on their own — the per-program binding directories that used to be built for them are legacy practice, and finding one in an old build script is not evidence that it is still required.

2.3 Copybooks (...H members)

- Include with `/copy qrpglesrc,membername` (lowercase, no library).
- **Every copybook starts with an include guard, and the guard comes FIRST** — immediately after `**FREE`, ahead of the purpose comment, the `MODIFICATIONS` log and the `BLDOBJ` directives:

```
**FREE
/IF DEFINED(TMXXXWRKH)
/EOF
/ENDIF
/DEFINE TMXXXWRKH

///
// Purpose of the member ...
//
// MODIFICATIONS:
// BS 08/20/26 Created.
// @@
// *> <BLDOBJ TEXT=" ... ">/>
///
```

The order matters because of what reads the member. `BLDOBJ` does not inline copy members today, but the SQL precompiler does: it builds an exploded copy of the source before compiling, with every `/copy` expanded in place. On a second inclusion the guard's `/EOF` stops that expansion — but only from the `/EOF` onward. **Anything above it has already been read**, on every inclusion.

Comments above the guard are harmless, which is why an existing member carrying its purpose block on top is not a defect worth a sweep. A **compiler directive** above it is not harmless: a `/DEFINE`, `/SET` or `/COPY` placed there is acted on again on the second inclusion, which is the thing the guard exists to prevent. Nothing marks that boundary, so the safe rule is the simple one — put the guard first, and there is no "is this line safe above the guard" question left to get wrong later.

The header comment is an ILEDoc `///` block, as it is in any other member.

Nothing today parses ILEDoc into a documentation repository — the one thing that reads it is the VS Code *Code for IBM i* plugin, which turns it into hover and content-assist help **on a symbol**, and a member-level block has no symbol to attach to. So it earns nothing right now.

Write it anyway. It is an opening and a closing line; it keeps copybooks looking like the rest of the source instead of a category with its own comment style; and if a documentation repository is ever built from these, the members are already annotated rather than needing a retrofit. Cheap now, and the only version of this that is expensive is the one done later.

The `BLDOBJ` directive goes inside that block, after the `@@`, exactly as it does in a program — see section 1.3. A copybook's header is not a different kind of header.

Copybooks whose header is plain `//` are not defects — convert one when you are in it for another reason.

- A copybook contains, in order: nested `/copy` of its dependencies, named constants, data-structure templates, then prototypes. Terminate each group with a separator rule.
- Everything a copybook publishes carries the copybook's **full** name as its namespace — `@GUSQCPY_*`, `gusqcpy_*_t`, `gusqcpy_*` — including the `...P` member's procedure bodies. See section 1.5 for the rule and its two exceptions.
- Shared IBM API prototypes live in the `QAPIH` (QUS*/QMH* APIs), `QILEH` (ILE CEE/QSN), and `QUIMH` (UIM) copybooks — extend those rather than redeclaring APIs inline.
- Document every prototype with a `///` doc block in **ILEDoc format**: one-line summary, `@param` per parameter, `@return` when a value is returned.

Two of these rules are Code for i's, not ILEDoc's — and the spec is the more permissive of the two, so reading it alone leads you wrong:

	ILEDoc	Code for i
tag case	any	lowercase only
return tag	<code>@return</code> or <code>@returns</code>	@return only

Write lowercase, and write `@return` without the s. Code for i is the only tool that reads any of this, so its restrictions are the effective standard whatever the spec allows.

@param names the variable — @param var Description. That is the ILEDocs syntax, and it stays.

Code for i renders the name twice. Its hover supplies the declared name and then prints the whole tag body after it, so a tag written to the syntax comes out as:

`@param pxForce pxForce Retrieve again even though it has been retrieved`

That is a defect in the extension, not a reason to drop the name — and dropping it is the poor practice the doubling invites. The extension binds tags to parameters **by position** and compares nothing, so a block that falls out of step with the parameter list silently re-attaches every later description to the wrong parameter. The name in the tag is the only thing that makes that findable. A feature request is open with the extension to strip a leading token when it matches

the parameter, and to leave it alone when it does not — which would keep the clean hover *and* the mismatch visible.

2.4 Declarations

- **Constants:** SCREAMING_SNAKE_CASE prefixed with @ (C-style), grouped by topic:
`dcl-c @QUIM_FNCKEY_ENTER 1;`, `dcl-c @GUSQCPY_SQLCODE_NODATA 100;`
- **Templates:** all-lowercase name with a `_t` suffix — "My Template Variable" becomes `mytemplatevariable_t` — declared `template qualified inz`, and namespaced with the publishing member's full name (section 1.5):

```
dcl-ds tmxxxwrk_usrlste_t    template qualified inz;  
    opt                    Int(5);  
    usrPrf                  Char(10);  
end-ds;
```

Instances drop the namespace, since they are local: `dcl-ds usrlsteE
likes(tmxxxwrk_usrlste_t) inz(*LIKEDS);`

- **External-file templates:** one per table, used for `like()` typing and fetch buffers. These are namespaced by the table, not by the copybook, so two copybooks may declare the same one behind an `/IF NOT DEFINED` guard:

```
dcl-ds tmusr_t    extname('TMUSR') inz(*EXTDFT) qualified template;  
end-ds;
```

- Type fields with `like( ... )/likes( ... )` against templates or other fields — avoid re-hardcoding lengths.
- **Casing:** camelCase for variables, procedures, and DS subfields; type keywords capitalized (`Char`, `Varchar`, `Int`, `Uns`, `Packed`, `Ind`, `Timestamp`); all other keywords lowercase — opcodes, declarations, and control words (`dcl-s`, `dcl-ds`, `ds`, `begsr`, `exsr`, `endif`) and BIFs (`%scan`, `%char`, `%trim`); figurative constants and compiler values uppercase (`*ON`, `*OFF`, `*BLANK`, `*NULL`, `*VARSIZE`, `*ESCAPE`, `*LIKEDS`).
- Align the type column within a declaration group; separate groups with blank lines or minor rules.
- Entry parameters take a `px` prefix (`pxTskName`); indicators declare `Ind`; cursors are named `csrXxx`.

2.5 Program structure

- **Entry:** the program's `dcl-pi PGMNAME` sits directly under a `// *ENTRY` marker, immediately after `/copy` of the program's own ...H copybook (the copybook carries the matching `dcl-pr ... extpgm('PGMNAME')`).

- **Mainline** is linear cycle-main code introduced by a `/// main() ///` doc block. It delegates to subprocedures and ends with `exsr exitPgm / return`.
- Use **subroutines** (`begsr`) only for mainline flow plumbing: `exitPgm` (set `*inLR`, close handles) and `*inzsr` (one-time init, cursor DECLAREs). All real logic goes in **subprocedures** (`dcl-proc`).
- Name subroutines — and subprocedures — `<verb><Object>` in camelCase: `getItemHeader`, `loadList`, `sndStsMsg`, `exitPgm`. A subprocedure that is published to other members through a copybook takes the namespace of section 1.5 in front of that name — `tmxxxwrk_sndStsMsg`.
- Every procedure carries a `///` doc block (ILEDoc, see 2.3): a one-line short description, then `@param` and `@return` lines as needed.
- Subprocedure skeleton:

```

///
// Loads a panel work with ... list
//
// @param hdl          Panel handle
// @param refreshDta Refresh the list data before loading
// @return *ON when the list is not positioned at the first entry
///
dcl-proc loadList;
  dcl-pi *N Ind;
    hdl          Char(8)      const;
    refreshDta    Ind          const options(*NOPASS);
  end-pi;
  // _____

  dcl-s rcdNam      Char(10);
  // _____

  ... body ...
  return endOpt <> @QUIM_LSTPOS_FIRST;
end-proc;

```

`dcl-pi *N`, locals after a minor rule, single exit where practical.

- Parameters are `const` unless returned; use `options(*NOPASS) / options(*OMIT: *NOPASS)` for optional parameters. Test them with:


```
if %parms() ≥ %parmnum(topEnt) and %addr(topEnt) <> *NULL;
```
- Prefer modern expression forms: `%char(%date(): *ISO0)`, `snd-msg`, and the `IN` operator — which has one rule attached to it.
- **ALWAYS** parenthesize an `in %list( ... )` test. Write it

```
if (needle in %list('stack1': 'stack2'));  
if not (needle in %list('stack1': 'stack2'));
```

A bare `if needle in %list(...)` on its own **evaluates correctly** — IBM gets that case right and there is nothing to fix in existing code that reads this way. The rule is not about the statement as written; it is about the statement as it will be edited.

not binds tighter than in. Put a **not** in front of an unparenthesized test and it negates the **needle**, then searches the list for *that* — it does not negate the result of the search. The comparison still runs, still returns something, and the program carries on with the wrong answer. No compile error, nothing to see in the source. The same exposure applies once the condition grows an **and** or an **or**.

So the parentheses are insurance against the edit, not a fix for the expression. `if needle in %list(...)` is correct today and becomes silently wrong the moment somebody adds a **not** in front or joins another condition to it — and that somebody will not stop to re-check operator precedence first. Parenthesizing always costs nothing and removes the trap.

The fragility is also how the unprotected form keeps creeping in. Nothing pushes back on it: it compiles, it runs, and it is right. Every other bad habit eventually meets a compile error or a failing test, so it gets trained out; this one never does, and spreads unopposed. That is why the rule is absolute rather than a preference — the parentheses have to come from the standard, because they will never come from the feedback.

Balance the siblings. Once one operand of an **and/or** is parenthesized, parenthesize the others so the alternatives read as parallel:

```
when (srcType in %list(@SRCTYPE_RPGLE: @SRCTYPE_SQLRPGLE))  
    or (%scan('RPG': srcType) > 0);  
  
if (objType = @OBJTYPE_FILE)  
    and (%subst(objName: 1: 3) in %list('PRT': 'WRK': 'WSN'));
```

The parentheses arrive for the `in %list()`, but leaving the other side bare makes the two sides look like different kinds of thing when they are the same kind of thing. This applies to **expressions** — a comparison, a BIF result being tested. A bare indicator or boolean variable is already atomic and gains nothing: write **when isNative** and **(inBldDir in %list(...))**, not **when (isNative) and (...)**.

Do not read this as C-style if (condition). The parentheses go around each operand, never around the whole condition — that is a different language's convention and RPG has no use for it.

There is also **no NOT IN operator**: `needle not in %list(...)` does not compile. The negation goes in front of the parenthesized test.

- Indent 2 spaces per level. Continuation lines indent 4 spaces; when breaking a parameter list, lead continuation lines with the `:` separator:

```
quim_displayPanel(hdl: fncKey: pnIDName: @QUIM_REDISPLAY_YES
    : errc0100: @QUIM_USRTSK_OLD
    : msgRefKey);
```

2.6 Error handling and messages

- Call APIs through the `qapi_*/quim_*/qile_*` wrappers with an `errc0100` structure (`likeds(qapi_errc0100_t); reset errc0100;` and set `bytPrv` before each call, then test `bytAvl`.
- Use `callp(e) + %error()` for tolerated command failures; escalate with `snd-msg *ESCAPE %msg( ... ) %target('*PGMBDY': 1);`.
- **"Qualified" and "fully qualified" are two DIFFERENT layouts, and the component order is reversed between them.** The terms differ by one word and are used in different contexts, which is the whole difficulty:

Term	Bytes	Layout	Where it appears
fully qualified	up to 21	LIB/NAME — library first, slash, name	what a user <i>types</i> ; what <code>%msg</code> accepts
qualified	20	NAME in 1–10, LIB in 11–20 — name first, no slash	what a QUAL parameter <i>delivers to the CPP</i> ; what the QMH* APIs take

So typing `MSGF(MSGLIB/MSGF)` hands the CPP `'MSGF MSGLIB '`. The library leads on the way in and the name leads on the way out. This is the same inversion that makes the `PGMXRF` convention work (§5): the two `QUAL` constants are written *CPP then command*, so the compiled reference arrives with the CPP in the object field and the command name in the library field.

This is not a battle worth fighting on the word alone. "Qualified" is too overloaded across IBM's own documentation, the API reference and the command reference for the adjective to carry the meaning by itself, and no house convention will change that. So do not depend on it: when the layout matters, **name it** — say "20-byte", or write `LIB/NAME`, or say "as the CPP receives it". Context is what disambiguates, so supply the context.

- **`%msg` takes the message file as a name — plain or FULLY qualified, never the 20-byte form.** It accepts `MSGF`, `MSGLIB/MSGF` and `*LIBL/MSGF`, so a variable holding one needs room for 21 characters. Name the file from a constant rather than a literal, and pick the right constant — `QAPIH` publishes two that look interchangeable and are not:

Constant	Value	For
@QAPI_QCPF_MSGF_NAME	'QCPFMSG '	%msg — a 10-byte name, trailing blanks ignored
@QAPI_QCPF_MSGF	'QCPFMSG *LIBL '	QMHSNDPM and friends — the 20-byte qualified form

Passing @QAPI_QCPF_MSGF to %msg is wrong even though it compiles and looks deliberate: %msg would read the whole 20 bytes as one name.

- Status messages go through a small `sndStsMsg` procedure (CPDA0FF / *STATUS); clear with a blank message when done.

2.7 Embedded SQL

- `exec sql` on its own line; the statement indented under it.
- SQL keywords UPPERCASE; table/column names lowercase; host variables :camelCase (qualified DS subfields allowed: :recUsr.usrPrf).
- Continuation style: **leading commas**, 2-space progressive indent:

```
exec sql
  SELECT a.usrPrf
        , a.grpPrf, a.usrName, a.emlAdr
  INTO :recUsr
  FROM tmusr a
  WHERE a.usrPrf = :usrPrf;
```

- Alias every table (a, b; z for innermost subselects) and qualify every column.
- Singleton reads end with `FETCH FIRST 1 ROWS ONLY`.
- Bulk reads use a `DECLARED` cursor + multi-row fetch into a `dim( ... )` DS array, then `CLOSE`; row count from `sqlEr3`:

```
exec sql
  FETCH csrUsrs FOR :@WRKUSRTM_USRS_MAX ROWS INTO :recUsrs;
```

- `DECLARE CURSOR` belongs in the mainline/*inzsr, **not** in a subprocedure (activation-group/scoping surprises).
 - Test `sqlcode` against named constants — @GUSQCPY_SQLCODE_SUCCESS, @GUSQCPY_SQLCODE_NODATA from GUSQCPYH — right after each statement; reset stale state (e.g. counts) before reuse.
-

3. CL / CLLE

QCLSRC/TEMPLATE.CLP is the canonical skeleton — start new programs from it.

3.1 Member type and header

- New CL is **CLLE** (.CLLE). .CLP remains only for legacy/OPM-specific cases.
- Header: purpose comment, **MODIFICATIONS**: log (comment lines continued with +), BLDOBJ directives, then:

```
PGM          PARM(&TSKNAME &#TOLOC ... &PGMXRF)
COPYRIGHT    TEXT('(c) Copyright Godsend Consulting, 2020-2026'
DCLPRCOPT    DFTACTGRP(*NO) ACTGRP(*CALLER)
```

3.2 Layout

- Keyword form always — **DCL VAR(&X) TYPE(*CHAR) LEN(10)**, never positional (except CMD/PARM positional prompts where conventional).
- **The layout is what F4=Prompt produces.** That is the whole rule, and everything below follows from it: prompting emits keywords in **command definition order**, aligns the columns, and wraps at keyword boundaries. When in doubt about a statement's shape, prompt it and keep what comes back.

The one thing prompting does not do is **indent** — SEU has no concept of block level, so the nesting in the table below is the house addition. (A VS Code formatter appears to apply the same rules *with* indentation, but driving it would mean a round-trip syntax check per line, which is far too slow to use in bulk.)

- **Columns follow standard CL prompting.** The command name occupies a 10-wide field followed by one blank, so the keyword column is always the command column + **11**:

	Command	Keywords	Wrapped keywords
Top level	14	25	27
Inside one control block	16	27	29
Inside two	18	29	31
...each level	+2	+2	+2

A subroutine is a control block. SUBR/ENDSUBR sit at column 14 and the body starts at 16 — which is a good reason to prefer SUBR over wrapping work in a DO, since it costs no extra level.

- **Labels** start in column 2 on their own line, at any depth.
 - **DCL with STG(*DEFINED)** is indented 2 under the variable it overlays, shifting the line and its continuations right by 2 like any other level.
 - **Comments start in column 1**, not the command column, and then follow the indentation level like anything else: column 1 at top level, 3 inside a subroutine, 5 inside a block within it. Continuations align under the comment text.
 - Stay within the 80-column SRCDTA limit (§1.4) at every level; deeper nesting spends the same budget.
- **Wrap on a keyword boundary.** Given MYCMD KWD1(xxx) KWD2(yyy) KWD3(zzz), break so the continuation *starts* with a keyword rather than splitting one across lines:

```
MYCMD      KWD1(xxx) KWD2(yyy) +
           KWD3(zzz)
```

Two things the column rules do not govern, so do not "correct" them:

- **A command name longer than 10 characters** — a fully qualified name such as HAWKEYE/DSPFILSETUP — pushes its keywords right of the nominal column. The field cannot hold it; nothing is wrong.
- **Large literals — SQL statements, built command strings — break the column rules on purpose, so the literal stays readable.** The command itself follows the normal columns; the *content* is laid out as what it is:

```
CHGVAR      VAR(&SQL) VALUE(' +
DELETE +
  FROM qtemp.' *CAT &OUTFILE1 *TCAT ' a +
  WHERE a.tudlib <> ''' *CAT &JC_LIB *CAT ''' +
                        ')
```

The CHGVAR sits at its proper column, the SQL starts in column 1 and is indented by SQL rules (§4.6), and the closing ')' returns to the continuation column. RUNSQL SQL(' takes the same shape.

Prefer this to concatenating a statement across CL continuations.

Wrapping SQL at CL keyword columns produces text that is legal and nearly unreadable — the statement's own structure disappears, and a reviewer cannot see the clauses. Readability of the literal outranks column discipline here, which is why the exception exists rather than being tolerated.

- Commands UPPERCASE; comments sentence case.

- **Block form for conditionals:** wrap IF/ELSE/WHEN bodies in DO ... ENDDO even when the body is a single statement, so a block can grow later without rewriting the conditional. Recognized exceptions where a one-liner is fine: the standard STDERR handler (IF COND(&STDERR) THEN(RETURN) — the most common one, written verbatim across programs), the occasional SELECT/WHEN one-liner, and legacy code.

3.3 Declarations

Order after PGM/COPYRIGHT/DCLPRCOPT:

1. **Parameters**, in PARM order.
 2. **DCLF** if the program reads a file (use OPNID).
 3. **Named-constant style variables** (DCL ... VALUE(...)) used as enums, e.g. &FROMTO_FRM ... VALUE(1).
 4. **Locals, alphabetical.**
 5. **&STDERR + the global MONMSG** (see 3.4) — always last.
- Structure overlays use STG(*DEFINED) with DEFVAR, indented two extra spaces under their base variable:

```

DCL          VAR(&JOBBD) TYPE(*CHAR) LEN(20)
DCL          VAR(&JOBBD_NAME) TYPE(*CHAR) STG(*DEFINED) +
              LEN(10) DEFVAR(&JOBBD 1)
DCL          VAR(&JOBBD_LIB) TYPE(*CHAR) STG(*DEFINED) +
              LEN(10) DEFVAR(&JOBBD 11)

```

- Qualified-object parameters are *CHAR 20 (name + library) split with overlays, as above.
- A parameter that can carry special values (*SAME, *N, generics) is named &#NAME; resolve it into a plain working variable &NAME before use.

3.4 Error handling — the STDERR pattern

Every program monitors globally and funnels to one handler:

```

DCL          VAR(&STDERR) TYPE(*LGL) VALUE('0')
MONMSG      MSGID(CEE0000 CPA0000 CPD0000 CPF0000 +
              CPI0000 CPP0000 MCH0000 QSH0000 PDM0000 +
              RNI0000 RNQ0000 RNS0000 RNX0000 RPG0000 +
              SQL0000 SQ20000 SQ30000 TCP0000 USR0000) +
              EXEC(GOTO CMDLBL(STDERR))

...
/* Handles thrown errors */
STDERR:
      IF      COND(&STDERR) THEN(RETURN)
      CHGVAR  VAR(&STDERR) VALUE('1')

```

```

/* Forward messages, resignaling escape message(s). */
    INCLUDE      SRCMBR(FWDMSGH) SRCFILE(QCLSRC)
    RETURN

```

- A module adds its own message prefix to the MSGID list — XFF0000 for the Cross Reference. Task Manager adds DEP0000 EXC0000 LOC0000 OBJ0000 TSK0000 UIM0000, six prefixes for one module — the standing exception of section 9.2, not the pattern to copy.
- The &STDERR flag prevents handler re-entry; FWDMSGH (QMHHMOVPM + QMHRSNEM) moves diagnostics to the caller and resignals the escape — callers see the original error, not a generic one.
- **Expected** conditions are monitored per-command and cleaned up immediately:

```

        CHKOBJ      OBJ(QTEMP/&OUTFILE) OBJTYPE(*FILE)
        MONMSG      MSGID(CPF9801) EXEC(DO)
            RCVMSG      MSGTYPE(*LAST) RMV(*YES)
            CHGVAR      VAR(&EXISTS) VALUE('0')
        ENDDO

```

Always RCVMSG ... RMV(*YES) a handled message so the joblog stays honest.

3.5 Subroutines and flow

- Mainline delegates with CALLSUBR; SUBR ... ENDSUBR blocks sit after the STDERR handler, each preceded by a one-line comment. Use RTNSUBR for early exit. Standard names: INITPGM, SNDSTMSG, plus task-specific ones.
- **MONMSG cannot follow CALLSUBR.** A monitor placed after a CALLSUBR does not catch what happened inside the subroutine — it attaches to the wrong statement. So a caller cannot trap an escape thrown by a subroutine it called.

Where the caller must react rather than let the escape reach the STDERR handler, **monitor each statement inside the subroutine and return a code:**

```

        CALLSUBR      SUBR(BLDONE) RTNVAL(&RC)
        IF             COND(&RC *EQ -1) THEN(GOTO CMDLBL(NEXTONE))

```

with the subroutine ending RTNSUBR RTNVAL(-1) on the failure path. The return variable must be a **4-byte signed integer** — TYPE(*INT) LEN(4).

Reset the variable before the first call in a loop. After a failed iteration it still holds -1, and a subroutine that succeeds without an explicit RTNVAL will not necessarily clear it.

Use this only for conditions the caller can act on — typically a property of the data being processed, where the next item may still succeed. A genuine error still

escapes to the STDERR handler; converting those to return codes discards the message that says what went wrong.

- Optional parameters: IF COND(%PARMS *GE n *AND %ADDR(&VAR) *NE *NULL).
- Work files live in QTEMP; inline SQL uses RUNSQL ... COMMIT(*NONE); file repositioning uses OVRDBF ... POSITION(*RRN &N) SECURE(*YES) followed by DLTOVR — always scope and remove overrides.
- **Create an outfile explicitly, and remove its size limit.** Do not let the command that fills it create it as a side effect:

```
CRTDUPOBJ OBJ(QADSPPGM) FROMLIB(QSYS) OBJTYPE(*FILE) +  
          TOLIB(QTEMP) NEWOBJ(XFPGMREF)  
CHGPF      FILE(QTEMP/XFPGMREF) SIZE(*NOMAX)
```

CRTDUPOBJ of the IBM-supplied template (QADSPPGM, QADSPOBJ, ...) gives the correct record format, and CHGPF SIZE(*NOMAX) removes the member size limit the template carries.

The CHGPF is not optional. The inherited limit is generous enough that development-sized data never reaches it, so omitting it fails only once the program is pointed at real volume — and it fails as a member-full condition on a QTEMP file, which explains nothing about the program that caused it. A limit that only bites in production is worse than one that bites immediately.

4. SQL (QSQLSRC)

Standards follow **RBUTL** practice exclusively (BSLIB's QSQLSRC contains ad-hoc test members that are not exemplars).

4.1 Member types and build

Ext	Contents
.SQLT	One table: DDL + labels + audit trigger + indexes
.SQLV	View(s) for one purpose
.SQLS	Everything else run as a script (UDFs, sequences, fix-up statements)

Every member is built by RUNSQLSTM via its directive header:

```
/*> RUNSQLSTM SRCFILE(&L/&F) SRCMBR(&N) COMMIT(*NONE) - <*/  
/*>           DFTRDBCOL(&O) DATFMT(*ISO) TIMFMT(*ISO) <*/
```

4.2 The schema-resolving compound block

DDL members are a single BEGIN ... END compound statement of **dynamic SQL**, so the target schema is resolved at run time (from DFTRDBCOL/*CURLIB) instead of being hardcoded:

```
BEGIN
  DECLARE tblNam  VARCHAR(10) DEFAULT 'TMTSK';
  DECLARE tblLib  VARCHAR(10);
  DECLARE tblRef  VARCHAR(21);
  DECLARE sqlStmt VARCHAR(5000);

  --Resolve the DFTRDBCOL/CURRENT SCHEMA/*CURLIB reference
  CREATE OR REPLACE TABLE
    bldobj_temp_resolve_dftrdbcol_schema_tmtsk
    (fld1 char(1));
  GET DIAGNOSTICS CONDITION 1 tblLib = DB2_ORDINAL_TOKEN_2;
  SET tblLib = TRIM(tblLib);
  SET sqlStmt = '
    DROP TABLE ' || tblLib
    || '.bldobj_temp_resolve_dftrdbcol_schema_tmtsk';
  EXECUTE IMMEDIATE sqlStmt;
  SET tblRef = tblLib || '.' || tblNam;
  ...
END;
```

- **The probe table's name ends with the MEMBER's name, and that is not optional.** The trick above works by creating a throwaway table with no schema qualifier, letting the resolved schema come back in the diagnostic, and dropping it again — so the name is a literal, and for years it was the *same* literal in every member.

Two members building at once in one library collide on it. The window is small and the failure is not: the second CREATE OR REPLACE replaces the first member's probe table, and the first member's DROP then removes a table the second is still relying on. Neither member did anything wrong and neither names the other in its error.

Append the member name —

bldobj_temp_resolve_dftrdbcol_schema_xfbld — and the collision is impossible rather than unlikely. There is nothing to coordinate and no lock to take.

Break both statements across two lines while you are there. The base CREATE line is already 78 characters in an 80-column file, so any suffix at all truncates it (§1.4), and a truncated CREATE OR REPLACE TABLE is exactly the balanced-quotes/C-compile-error signature described there. The two-line form is used in every member regardless of how short its name is, so nobody has to measure:

```
CREATE OR REPLACE TABLE
    bldobj_temp_resolve_dftrdbcol_schema_xfbld
    (fld1 char(1));
...
SET sqlStmt = '
    DROP TABLE ' || tblLib
    || '.bldobj_temp_resolve_dftrdbcol_schema_xfbld';
```

- Build each statement into `sqlStmt` and EXECUTE IMMEDIATE it.
- Before re-creating a table, loop the `qsys2.sysindexes` / `qsys2.systtrigger` catalogs and DROP existing indexes and triggers (FOR row AS csr CURSOR FOR ... DO ... END FOR;).
- Members must be **re-runnable**: CREATE OR REPLACE everywhere; identity columns re-seeded at the end with ALTER ... RESTART WITH from MAX(id) + 1.

4.3 Tables (.SQLT)

```
CREATE OR REPLACE TABLE ... (
    task_id          FOR COLUMN tskid          BIGINT
    GENERATED BY DEFAULT
    AS IDENTITY (START WITH 1 INCREMENT BY 1 CYCLE)
    PRIMARY KEY
, task_name         FOR COLUMN tskname        CHAR(10)
    NOT NULL DEFAULT UNIQUE
, task_status       FOR COLUMN tsksts        CHAR(1)
    NOT NULL DEFAULT '0'
    CHECK(task_status BETWEEN '0' AND '9')
...
) RCDFMT r' || tblNam;
```

- **Long descriptive names** with FOR COLUMN short (≤ 10) system names, both lowercase.
- **Avoid SQL reserved words as column names, even where they compile.** Db2 enforces the [reserved word list](#) by context, so a column named SEQUENCE or POSITION is accepted today — the word has no meaning in that position, so the parser allows it. Do not rely on that. IBM reserves the right to add to the list at any time, including in PTFs against a GA release, and has done so in ways that broke working code. A name that compiles this year is not a name that compiles next year.
- **The defence is a domain prefix, not the list.** You cannot keep up with the list, so make the question moot: name every column for what it belongs to — `libl_position`, `merge_sequence`, `build_status`, `environment_type`. IBM will never reserve those. This is the same move IBM itself makes in `QSYS2.LIBRARY_LIST_INFO`, whose column is `ordinal_position` rather than `position`. Note that *compound* is not by itself sufficient — `SYSTEM_USER` and

CURRENT_DATE are reserved — it is the domain prefix that makes a collision implausible. Prefer this to a delimited identifier, which merely postpones the problem into every statement that reads the column.

- Identity **BIGINT** primary key named **<entity>_id**.
- **NOT NULL DEFAULT** on business columns; **CHECK** constraints where the domain is enumerable. Enumerated values are **numeric, not alpha** — see section 10.
- **Audit columns**, all **IMPLICITLY HIDDEN**: **audit_timestamp** (row change timestamp), **audit_job_number/user/name**, **audit_current_user**, plus matching **create_*** columns.
- A **BEFORE INSERT OR UPDATE ... FOR EACH ROW MODE DB2ROW** trigger, named **<table>T1**, normalizes keys (**LTRIM(UPPER(...))**), fills defaults, and populates the audit/create columns; new rows are detected by the identity column being **NULL** in the **OLD** row.
- **RCDFMT r<table>** so record-level access sees a stable format name.
- **LABEL ON TABLE, LABEL ON COLUMN ... TEXT IS** (full text) **and LABEL ON COLUMN ... IS** (column headings) for every column.
- Indexes named **<table>9, <table>8, ...** (9 = unique/primary alternate key, descending from there), each with **LABEL ON INDEX ... IS 'By <keys> (u)'**.

4.4 Views (.SQLV)

- **CREATE OR REPLACE VIEW <ref>(col, ...)** AS with an explicit column list.
- Structure complex logic as **WITH CTEs**; name CTE columns explicitly.
- Cap recursive/**CONNECT BY** depth explicitly (e.g. **AND a.recid ≤ 250**) and document why in the header comment.
- **LABEL ON TABLE <view> IS '...' after creation.**

4.5 Routines (UDFs/procedures, usually .SQLS)

- **CREATE OR REPLACE FUNCTION** with a **SPECIFIC** name per overload (**char8ToDate, dec8ToDate**).
- Declare behavior: **DETERMINISTIC, CONTAINS SQL/READS SQL DATA, RETURNS NULL ON NULL INPUT, NO EXTERNAL ACTION**.
- Handle bad input via a scoped **DECLARE CONTINUE HANDLER FOR SQLEXCEPTION** and return **NULL** rather than erroring.
- **COMMENT ON SPECIFIC FUNCTION** and **COMMENT ON PARAMETER** for every routine — these surface in catalogs and client tools.
- Parameter defaults (**DEFAULT 'YMD'**) instead of overload explosions.

4.6 Style

- Keywords **UPPERCASE**; identifiers **lowercase**; 2-space indent; leading commas on continuation lines (both column lists and select lists).
- Alias every table; qualify every column reference.

- -- for inline comments inside SQL bodies; /* */ for the member header.
- **A CASE search-condition may not contain a subquery.** From the *SQL Reference*, under **search-condition** in the CASE expression syntax:

The search-condition must not include a subquery in an EXISTS or IN predicate.

So CASE WHEN EXISTS (SELECT ...) THEN ... END — idiomatic in most dialects and the obvious way to flag a row — is simply not available here. Neither is CASE WHEN x IN (SELECT ...).

The diagnostic names the wrong thing, which is what makes this expensive.

Db2 reports it as SQL0104:

Token EXISTS was not valid. Valid tokens: <IDENTIFIER> <INTEGER> <CHARSTRING> <GRAPHSTRING>.

Nothing about CASE, nothing about subqueries — just a token the parser did not want, and a list of value-expression starters that reads as though the EXISTS keyword itself were unrecognized. In **dynamic** SQL there is no precompile to catch it either, so the first sign is a PREPARE that fails at run time.

Rewrite as a LEFT JOIN against a derived table and test the joined column for NULL:

```
SELECT a.name
      , CASE WHEN d.nam IS NULL THEN '1' ELSE '0' END
FROM xfxrf a
LEFT JOIN (SELECT DISTINCT z.environment AS env
           , TRIM(z.WHFNAM) AS nam
           FROM xfxrf z WHERE z.environment = ?) d
ON d.env = a.environment
AND d.nam = TRIM(a.WHPNAM)
```

This is usually the better plan anyway — one pass instead of a correlated subquery per row — so it is not a workaround being tolerated. Restrict the derived table by the same predicate the outer query uses, and remember the marker it adds comes **first**, because FROM is parsed before WHERE.

The other rewrite is a scalar subquery in the SELECT list, tested by the CASE as a column. The restriction is on a subquery *inside* the search-condition, not on subqueries in the statement, so lifting it one level out is enough:

```
WITH r (name, hasref) AS (
  SELECT a.name
        , (SELECT 1 FROM xfxrf z
           WHERE z.environment = a.environment
           AND z.WHFNAM = a.WHPNAM
```

```

        FETCH FIRST 1 ROWS ONLY)
    FROM xfxrf a
    WHERE a.environment = ? )
SELECT a.name
      , CASE WHEN a.hasref IS NOT NULL THEN '1' ELSE '0' END
FROM r a

```

Which of the two depends on how many rows the OUTER query returns, not on how big the table is. The LEFT JOIN reads its derived table once, whole, before the first row can be returned — a fixed cost paid even when the outer query returns four rows. The correlated subquery runs once per returned row and probes an index each time.

So: a **join** where the outer query returns a large fraction of the derived table, and a **scalar subquery** where it returns a handful of rows out of many and the correlation is on an indexed column. An inquiry that reads one object's references out of a hundred thousand is squarely the second, and the DISTINCT the join needs is a sort on top of the scan.

Both forms are correct; the wrong one is a full read and a sort before the panel draws.

- **Test sqlcode after PREPARE and OPEN, and treat a failure as a failure.** A dynamic statement that will not prepare fetches no rows, so code that only checks the FETCH reports a broken query as an empty result — and an empty result looks like a data problem, which sends the next person to the wrong place entirely. Escape with the operation and the SQLCODE in the text; CPF9898 carries arbitrary text and needs no message file of your own.
- **LIKE and REGEXP_LIKE trade an index against a readable predicate.** Both match a name; the choice is not a matter of taste and it is worth making deliberately.

LIKE can drive an index. The *Database Performance and Query Optimization* manual lists the predicates the optimizer will match against an existing index — =, <>, LIKE, IN. REGEXP_LIKE appears nowhere in that book, and as a predicate over the column it defeats a keyed index the way TRIM(column) does. A derived key index cannot rescue it: the pattern is a run-time value, so there is no fixed expression to index.

REGEXP_LIKE says what it means. LIKE does not ignore trailing blanks the way = does, so matching an exact name against an untrimmed CHAR(n) means padding the pattern out to the column width — correct, fast, and opaque six months later. The regular expression anchors instead:

	LIKE	REGEXP_LIKE
exact	pattern padded to the column width	'^NAME *\$'
generic	'NAME%'	'^NAME'

REGEXP_LIKE *searches* rather than matches — the *SQL Reference* says it "searches for a regular expression pattern in a string" — so ^ is what makes the name mean the start of the column, and an unanchored end is what makes a generic name generic. '^NAME.*\$' adds nothing to '^NAME'. On the exact form the quantifier matters: ' *\$' allows the blanks the column carries, while ' ?\$' allows one and matches nothing.

One thing to know before choosing it: it requires ICU — "*In order to use the REGEXP_LIKE predicate, the International Components for Unicode (ICU) option must be installed.*" A parameter marker needs no CAST; function resolution types an untyped argument from the parameter it maps to.

Decide on the row count, not the principle. Against a few thousand rows per selection the scan is free and the clearer predicate wins outright. Against a few hundred thousand the index is the difference between a list and a wait. Measure the table before assuming either.

For a NAME, BETWEEN beats both — it settles the trade instead of making it.

Exact or generic-prefix, a name match *is* a range, and a range over the bare column is the strongest thing the optimizer can be handed:

	LIKE	REGEXP_LIKE	BETWEEN
exact	pattern padded to the column width	'^NAME *\$'	both ends NAME
generic	'NAME%'	'^NAME'	NAME+x'00' ... NAME+x'FF'
drives an index	yes	no	yes

It wins on all three of the things the paragraphs above are weighing:

- **No padding, so no opacity.** BETWEEN compares blank-padded like = does, so an exact name against an untrimmed CHAR(n) is just the name at both ends. That was the whole reason to reach for the regular expression.
- **No metacharacters, so no hazard.** An underscore in a LIKE pattern matches *any single character*, so a name containing one silently over-selects — and nothing about the result says so. A range has no pattern to escape.
- **One predicate in static and dynamic SQL alike.** = for exact and LIKE for generic means an operator that changes at run time, which a built statement

can do and an embedded one cannot without being written twice. **BETWEEN** is the same text either way.

Seed the ends and move the prefix over the top:

```
// Generic - prefix, then the low and high seeds
objLo = *LOVAL;
objHi = *HIVAL;
%subst(objLo: 1: prefixLen) = %subst(nam: 1: prefixLen);
%subst(objHi: 1: prefixLen) = %subst(nam: 1: prefixLen);

// Exact - a range of one
objLo = nam;
objHi = nam;
```

Type both ends as the column's own type and length, so the comparison needs no cast.

Reach for LIKE or REGEXP_LIKE where the match is genuinely not a prefix — text embedded anywhere in the value, alternation, a character class. Those are not ranges and no index was going to help.

4.7 NULL handling

- **Use COALESCE, not IFNULL.** Db2 documents them as equivalent for the two-argument case, and for a scratch query either is fine. In production queries **IFNULL** has been observed to send the optimizer off into a bad plan on complex statements — not on large *data*, but on structurally complex ones (deep CTEs, recursion, lateral joins). No mechanism is claimed here; the point is that the two are interchangeable in meaning, so there is nothing to weigh against standardizing on the one that has never caused trouble. **COALESCE** is also the SQL-standard spelling and takes more than two arguments, so a fallback chain does not have to be rewritten when a third source appears.
- **Assume every column from an IBM i Service is nullable unless documented otherwise.** The services return NULL rather than blanks for absent text — **TEXT_DESCRIPTION**, **OBJTEXT** and their like — which collides with the **NOT NULL DEFAULT** convention of section 4.3. Coalesce at the point the value enters a table.
- **Test for null and blank together** when a value is "missing or empty", because a service may express the same idea either way:

```
CASE WHEN COALESCE(TRIM(a.text), '') <> '' THEN a.text
      ELSE COALESCE(b.text, '') END
```

- Weigh the blast radius, not the row. A **NOT NULL** violation fails the whole statement, and where that statement is one step of a build that escapes on error, a single unguarded row takes down the entire run — on data nobody controls. That asymmetry is why the guard goes in by default rather than after the first failure.

- **NULLS FIRST and NULLS LAST do not exist on ORDER BY here, and are not needed.** The *SQL Reference* gives the statement-level `order-by-clause` as a `sort-key-expression` with `ASC` or `DESC` and nothing else. The two keywords are real, which is what makes this worth writing down — they belong to **OLAP window ordering** and to **partitioning keys**, so they are findable in the manual, in a syntax diagram that is not the one you want.

The behaviour is already what a `NULLS LAST` would ask for. From the *Queries* chapter: **"Null values are sorted high."** So ascending puts nulls last and descending puts them first, with nothing to write.

Where a sort has to treat a value as "no position" rather than as a low one, `NULLIF(column, 0)` does it on its own:

```
ORDER BY NULLIF(a.libl_position, 0), a.name
```

Rows with `0` sort to the end because they became null, and the rest sort normally. Adding `NULLS LAST` to that is a syntax error, and adding `ASC` first does not rescue it — the clause is not accepted in this position at all.

5. Commands (QCMDSRC)

```
/* MODIFICATIONS: +
   BS 08/02/25 Created. +
   @@ */
/*> CRTCMD CMD(&O/&ON) SRCFILE(&L/&F) SRCMBR(&N) PGM(TMACRUSR) <*/
/*> <BLDOBJ EOF/> <*/
/* (c) Copyright Godsend Consulting, 2025-2026 */
   CMD          PROMPT('Add User') MSGF(TMMMSG) HLPID(*CMD) +
               HLPPNLGRP(ADDUSRTM)
```

- Every command declares `PROMPT`, `MSGF(TMMMSG)` (or product message file), `HLPID(*CMD)`, and `HLPPNLGRP(<own name>)` — help is not optional.
- **PARM** conventions:
 - `EXPR(*YES)` on user-enterable parameters.
 - Mixed-case text parameters: `TYPE(*CHAR) LEN(n) CASE(*MIXED)`, **fixed length**. See below before reaching for `VARY`.
 - Special values via `SPCVAL((*NONE ' '))` mapping to storable values.
 - Selection lists via `CHOICE(*PGM) CHOICEPGM(<...CHC program>)`.
 - Shared-CPP commands pass their mode as `PARM KWD(MODE) TYPE(*CHAR) CONSTANT(' *ADD')`.

- **A special value that replaces the WHOLE parameter goes in SNGVAL.** VALUES and SPCVAL are **per element**. On a single, base-data-type parameter that is exactly right and SPCVAL is where special values go — the element *is* the parameter, so SNGVAL has nothing to add. The distinction bites on every **compound** parameter:

Parameter	Why it is compound
MAX() greater than 1	a list of elements
TYPE(ELEM-label)	a list of parts, one ELEM each
TYPE(QUAL-label)	a qualified name, one QUAL each

On any of those, SPCVAL((*SAME)) says *each entry may be *SAME*, so LIBL(*SAME QGPL) and LIBL(*SAME *SAME) are both accepted. SNGVAL ("single value") is the one that means **this value may only appear alone**, which is what *SAME, *NONE, *ALL and their like almost always want.

- **A list of BASE-DATA-TYPE elements arrives as a 2-byte binary count followed by the elements, each at its FULL declared length.** Fixed-length *CHAR, *NAME and decimal elements are padded to LEN(), so the stride is constant and the structure maps directly — in RPG as a dim() array behind the count, in CL as %BIN plus %SST or a STG(*DEFINED) overlay:

```
dcl-ds xxx_libl_t  template qualified inz;
    count          Int(5);
    library         Char(10) dim(250);
end-ds;
```

A VARY element does not lay out this way — each carries its own length — so do not assume a constant stride for one.

- **MAX() over ELEM or QUAL is a DIFFERENT structure. Do not reach for a dim() array.** The elements are not contiguous; the parameter opens with a count and then an **offset table**, and each entry carries its own element count before its components:

Offset	Content
1–2	number of entries passed (2-byte binary)
3 ...	one 2-byte offset per entry, relative to the start of the parameter
<i>per entry, at its offset</i>	2-byte count of elements supplied, then the components at their declared lengths

QCLSRC/INF2XLSX.CLLE is the worked reference — PARM KWD(ATR) TYPE(EATR) MAX(50) with ELEM lengths 50 and 256. It walks the offset table with a based pointer, advancing %OFS(&ATROFSPTR) by 2 per entry and setting the entry pointer to %OFS(&ATRPTR) + &ATROFS. The declared sizes confirm the layout: entry = 2 + 50 + 256 = 308; 50 offsets × 2 = 100, plus 50 × 308 = 15,400, giving the LEN(15502) = 2 + 15,500 on the receiving variable.

The offset table is the authority on where an entry is — never compute it. The entries have been observed loaded **back to front**: the first offset points at the *last* entry in storage, so walking the offsets forward walks backwards through the array. That is an observation rather than documented behaviour, which is exactly the point — the arrangement is not something to depend on in either direction, and reading the offsets is what makes it not matter. Stride arithmetic would land on the wrong entry even with every length correct.

Confirm the layout before writing against it. The above is verified for ELEM; a QUAL inside a list is not, and IBM documents these structures per parameter type rather than as one rule. The widths in particular are easy to get wrong — 2 bytes, not the 4 the shape suggests.

- **The CPP must bound every loop by the COUNT**, never by the declared dimension. The command materialises the elements it was given, not MAX() of them, so the array past the count is not blank and is not reliably even this parameter. Probed with one *SAME in each of two lists followed by a MODE constant:

```
LIBL    n=00001 [1]=*SAME [2]= <attr>*SAME
MRGENV  n=00001 [1]=*SAME [2]=*RSTD
MODE    = *RSTD
```

MRGENV element 2 returned *RSTD — the value of the **MODE parameter that follows it**. Reading past the count reads adjacent parameter data, which looks like a plausible name and would be written to the table as one. **What lies past the count is the call stack**, so this is not a tidy-up: it is reading storage that belongs to something else, and the value it returns changes with the caller.

- **RSTD(*YES) restricts which values are accepted, not how many**, so it is no substitute for SNGVAL — MAX() governs the count independently.
- **What DFT may be, in order.** A default has to be:
 1. valid for the data type — length and precision;
 2. inside RANGE and any other restriction;
 3. one of VALUES, where RSTD(*YES) is specified;

...or one of SPCVAL, which overrides all of it. SPCVAL is special in the literal sense: a value declared there is available to DFT whatever else the parameter says, including when it resolves to something the restrictions would reject.

```
PARM  KWD(NBR) TYPE(*INT) RANGE(11 22) SPCVAL((*JUNK -1)) DFT(*JUNK
```

That is legal and passes `-1` to the CPP. `DFT(-1)` on the same parameter is not, because `RANGE` governs values written literally.

So a default that will not fit the restrictions is not a reason to loosen them — it is a reason to name the value in `SPCVAL`, which is also how the CPP ends up receiving something it can recognize as "the user said nothing" rather than a number in the valid range that has to double as a flag.

- **VALUES is the parameter's real domain. Anything that is not a member of it goes in SPCVAL** — even when it looks like it belongs, and IBM special values look like they belong.

An object type parameter is the clean example. `*CMD`, `*DTAARA`, `*FILE`, `*PGM` and `*SRVPGM` are IBM object types: a subset of a real enumeration, so they are **VALUES**. A `*DFT` or `*ALL` meaning *do not filter on type* is not an object type at all, however much the asterisk makes it look like one — it is a pseudo-value, and it belongs in `SPCVAL` where it can resolve to the blank the CPP actually wants.

The tell is not the leading asterisk, which both kinds carry. Ask whether the value **names a member of the set the parameter is about**. `*FILE` names an object type. `*DFT` names a decision about filtering.

Getting it wrong compiles and works, which is why it persists — `IPOXINQ` carried `*OBJ` in **VALUES** for twenty years. The cost is that the prompt and the help present a pseudo-value as though it were one more object type, and the parameter stops documenting its own domain.

- **CONSTANT requires MAX(1)** — CPD6228. A list parameter cannot be locked with it, so a shared-CPP command that must pass a list parameter it has no use for declares it with `SNGVAL` and a `DFT` instead.

VARY(*YES *INT2) is for long text, and it is a cost to justify. A varying parameter arrives as a 2-byte length followed by the data. RPG receives that as a `Varchar` and nothing more is needed. **CL has no varying type**, so a CL CPP must take the parameter apart by hand — a base variable plus two `STG(*DEFINED)` overlays, and an `%SST` to extract:

```
DCL          VAR(&RCPV) TYPE(*CHAR) LEN(10242)
DCL          VAR(&RCPL) TYPE(*INT) STG(*DEFINED) LEN(2) +
              DEFVAR(&RCPV 1)
```

```

DCL          VAR(&RCPD) TYPE(*CHAR) STG(*DEFINED) +
              LEN(10240) DEFVAR(&RCPV 3)
...
CHGVAR      VAR(&RCP) VALUE(%SST(&RCPD 1 &RCPL))

```

Three declarations per parameter, and most CPPs are CL. So:

Field	Form
Tens of bytes — a description, a name, a subject	fixed TYPE(*CHAR) LEN(n) CASE(*MIXED)
Hundreds of bytes or more — a note body, a recipient list, an SQL statement	add VARY(*YES *INT2)
Any length, where the CPP is RPG and the value maps to a VARCHAR column typed with Like()	add VARY(*YES *INT2) — one shape from command to table

The test is whether the CPP genuinely benefits from being **told** the length. At 50 bytes it does not: %TRIMR is free and the buffer is trivial, so the overlays buy nothing. At 5,000 or 10,000 they buy something real.

Existing VARY parameters stay as they are — not violations, and not tracked for revision. The clumsiness is its own filter: nobody writes three overlay declarations per parameter by accident, so where VARY is already in place someone had a reason. Changing it would land on a working CPP for no behavioural gain. This rule governs new parameters.

This rule was rewritten 08/17/26. It previously read VARY(*YES *INT2) flatly for every mixed-case text parameter, generalized from four commands — three of which share one **RPG** CPP (TMACRUSR, 50-byte parameters against VARCHAR(50) columns), while the fourth (SNDXLSX, 10240/5000/255) is CL and pays nine DCLs to receive three parameters. Meanwhile a dozen other commands already used fixed *CHAR with CASE(*MIXED). The stated rule was the minority practice.

- Every command ends with the hidden cross-reference parameter:

```

QPGMXRF:    PARM      KWD(PGMXRF) TYPE(QPGMXRF) PGM(*YES)
              QUAL     TYPE(*NAME) LEN(10)  CONSTANT(<CPP name>)
              QUAL     TYPE(*NAME) LEN(10)  CONSTANT(<command name>)

```

The CPP receives it as a 20-byte qualified name (pxPgmXr-f / &PGMXRF).

This parameter is what makes command usage cross-referenceable, and it is not optional. No compiler records "this program used command X" — command invocation leaves no reference behind. PGM(*YES) on a qualified name does: the compiler records it as a **program reference**, and because the two constants are *CPP first, command second*, the reference arrives with the CPP in the object field

and **the command name in the library field**. A cross reference build detects that pairing and turns it into a command reference.

Omit the parameter and the command becomes invisible to the cross reference — "what uses this command" answers nothing, and nobody can tell whether it is safe to change. Get the two constants the wrong way round and the pairing does not match, with the same result and no error to say so.

There is no recovering this after the fact. Command invocation leaves no compiled reference of any kind, so nothing can be derived later — establishing usage would mean parsing source, which is not a thing anyone is going to do. The parameter at build time is the only record that will ever exist.

And the failure is in the dangerous direction: a command with no callers in the cross reference looks *unused*, which is indistinguishable from one that is used everywhere by programs the tool could not see. This convention is the only thing standing between that and a deletion.

6. Panel groups (QPNSRC)

- Header: `. * purpose comment, MODIFICATIONS: log, . *> BLDOBJ directives.`
- Open with `:PNLGRP ENBGUI=YES.` and `:COPYR.(c) Copyright Godsend Consulting, <years>.`
- **Reuse shared help:** `:IMPORT NAME='*' PNLGRP='TMHLP'.` and `:IMHELP` the shared modules (`TMHLP/<PARAM>/REQ`) instead of retyping parameter help.
- **:HELP titles stay on one line:** the descriptive text that follows `:HELP NAME='...'` cannot wrap to the next line, and must not be abbreviated to fit — let the line run long (up to the 134 hard limit) rather than shorten the wording.
- Command-help members define this fixed set of help IDs, in order:
 1. `'<CMD>/ALL'` — aggregate of all sections (for full-command help). **It carries no title text and no prose of its own** — just `:IMHELP` for each section, in the order below:

```
:HELP NAME='DSPENVXF/ALL' .  
:IMHELP NAME='DSPENVXF' .  
:IMHELP NAME='DSPENVXF/ENV' .  
:IMHELP NAME='DSPENVXF/OUTPUT' .  
:IMHELP NAME='DSPENVXF/COMMAND/EXAMPLES' .  
:IMHELP NAME='DSPENVXF/ERROR/MESSAGES' .  
:EHELP.
```

:IMHELP is the only way to pull one help module into another — there is no include tag. Put a title after the . on the ALL tag and CRTPNLGRP answers:

CPD5A8F More than one line of text specified for HELP tag.

which names the symptom rather than the cause: the title is one line of text and whatever follows becomes a second, so any stray line under the tag surfaces as this message.

2. '<CMD>' — extended description
3. '<CMD>/<PARM>' — one per parameter, titled <Prompt> (<KWD>) - Help
4. '<CMD>/COMMAND/EXAMPLES' — numbered examples in :XMP. blocks with a prose explanation per example
5. '<CMD>/ERROR/MESSAGES' — message text pulled live via &msg(MSGID,MSGF,*LIBL,nosub)., one :DL COMPACT. list per message type, each introduced by a highlighted paragraph rather than a list term:

```
:P. :HP3.*ESCAPE &msg(CPX0006,QCPFMSG):.EHP3.
:DL COMPACT.
:DT.ENV0001
:DD.&msg(ENV0001,XFMSG,*LIBL,nosub).
:EDL.
```

- **A period in column one is a control word — use &period. instead.** UIM reads column one for markup before it reads the line as text, so a line beginning with . is a control word (.im, and .* for a comment). The compiler rejects the rest of the word as one it does not know:

```
.savf .bin . .svf .dta .data
CPD5A02 Control word SAVF not defined.
```

Which names a control word nobody wrote, on a line that is plainly text — and inside an :XMP. block, where literal text is exactly what was asked for. The block does not suspend control word processing; nothing does.

The manual supplies a symbol for precisely this: &period. *"Creates a period (.) in the text. This symbol is used when a period is needed in column one of the text without triggering control word processing."* Only the leading period needs it, and it resolves to one character, so the column alignment of a sample survives:

```
&period.savf .bin . .svf .dta .data
```

Prefer this to indenting the line, which fixes the compile by changing the text.

Where the text is a list, tag it as one and the question does not arise. A run of values in an :XMP. block is the usual way a column-one period gets written in the

first place, and the block was rarely the right tag for it: a `:UL./:LI.` puts the text after the tag, so no item starts in column one. Reach for `&period.` when the text genuinely is a sample.

Prefer `:UL.` to `:OL.` The numbers an ordered list draws are a claim that the position of an item means something — a step, a rank, a count the prose refers back to. A list that is merely *in* an order does not need them, and getting that wrong is not neutral: it invites the reader to look for a significance that is not there. Order the items and say so in the lead-in sentence instead.

Picking the list is the same test as everywhere else in this section. `:PARML.` is for the values a **parameter** may be set to, so a list of things the user never types is not one, however value-like it looks. `:DL.` pairs a term with a description, so it earns its place only where the descriptions are real — a list whose items are self-explanatory would need five invented `:DD.s` to carry one that is not, which is worse than the plainer tag.

`&slr.` is the same thing for a right slash, and it is worth knowing before it is needed: a `/` in column one *compiles*, then breaks the member when the source is part of a batch input stream, because the reader takes it for `/*` or `//`. That failure arrives nowhere near the panel group.

- **Let the tags do the formatting.** UIM renders text according to the tag that contains it — lists, definition terms, parameter values and keywords all carry their own appearance. **Tag the text correctly and the highlighting takes care of itself.**

`:HPn.` is for **headings**, plus the occasional term or special reference inside running prose. Reaching for it to make tagged content *look* right is a sign the wrong tag was used — and it is **not valid inside `:PV.`, `:PK.` or `:DT.`** at all, because those take a bare term and already render it.

The two forms that are idiomatic here, and the only ones needed in practice:

```
:P.:HP2.Example 1: Simple Command Example:EHP2.
:P.:HP3.*ESCAPE &msg(CPX0006,QCPFMSG):EHP3.
```

Getting the invalid nesting wrong compiles nowhere — `CRTPNLGRP` rejects it with `CPD5A92 PK tag not allowed in text for DT tag.`

— and it is easy to write by analogy from HTML or Markdown, where nesting emphasis inside anything is fine:

Want	Write	Not
A variable value	<code>:PT.:PV.environment-name:EPV.</code>	<code>:PV.:HP2....:EHP2.:EPV.</code>

Want	Write	Not
A special value	:PT. :PK. *ALL:EPK.	:PT. :PV. *ALL:EPV.
The default value	:PT. :PK DEF.*YES:EPK.	—
A list of the values a parameter takes	:PARML. with :PT./:PD.	:DL. with :DT./:DD.
A message-group heading	:P. :HP3.*ESCAPE ...:EHP3.	:DT. :HP4....:EHP4.

:PARML and :DL are two different lists, and only :PARML accepts value tags. :PT. is a parameter *term* and takes :PK. and :PV.; :DT. is a definition *term* and takes neither. So anything enumerating what a parameter or a field may be set to is a :PARML, every time — including help written for a **panel** field rather than a command parameter, which is where the wrong one gets reached for, because the content is prose and the :DL looks like the plainer choice.

:DL/:DT/:DD is for plain terms with no value tagging. In practice that is the ERROR/MESSAGES section, where :DT. carries a bare message id and :DD. its &msg(...) — and that one stays as it is.

The restriction is on the TERM, not on the whole list. :DT. is the tag that refuses; :DD. is running text and takes :HPn. quite happily — BLD0BJ.PNLGRP has :DD.The specified :HP2.ALWCMDRTV:EHP2. parameter value and compiles in all three repos. So a :DL carrying emphasis in its descriptions is not the defect this rule is about; do not "fix" one.

:PV. is a *variable* — something the user substitutes, like **environment-name**. :PK. is a *keyword* — a literal special value such as *ALL. Only :PK. takes the DEF attribute, which underlines the value to mark it as the parameter default; :PV. has no equivalent, so a default that is a variable cannot be marked and does not need to be.

- Work-with (inquiry) panels declare, in separate banner-labeled sections: classes (cls<type><len>, e.g. clsname10, clsyesno with :TL. truth labels), variables (lowercase, matching the RPG DS subfields), :VARRCDs (header, detail, csrrcd, cmdlin), :LISTDEFs (dtllist with EMPHASIS columns), conditions, and key lists — the names must match the @..._PNLID / @..._VARRCD / @..._LSTNAM constants in the RPG ...H copybook.
- **DEPTH counts the separator lines, not just the content.** The manual: "*The depth of the area in lines, including separators if any are specified.*" So a :DATA area with

BOTSEP=space holding one line of content needs DEPTH=2 — at DEPTH=1 the separator consumes the whole area and CRTPNLGRP says

CPD5B8A DATAI will not fit in the body of this DATA area.

which reads as though the *item* were too wide. It is not about width at all. Check the separators before touching the item.

WRK0BJTM's description area is the reference: one line of text, BOTSEP=space, DEPTH=2. It looks like it has a spare line and it does not.

COMPACT is not cosmetic — without it every item costs two lines. The manual: *"If COMPACT is specified, no blank lines are left between column headings and all items and outer groups in the area. If COMPACT is not specified, a blank line appears after the area column headings and between all items and outer groups."* So a LAYOUT=1 area holding four items needs seven lines without it and four with it, and CRTPNLGRP reports the shortfall as

CPD5B99 No space left in this DATA area for remaining items.

naming neither the blank lines nor the keyword that would remove them. **Put compact on every data area** unless the spacing is wanted, and then count the depth as one line per row.

The arithmetic, once and for all — an area needs:

rows	items ÷ LAYOUT, rounded up
+ gaps	rows – 1, unless compact
+ 1	if MAXHEAD=1
+ 1	per :TOPINST
+ 1	if BOTSEP=space

Only one area on a panel may code DEPTH= '*', and DEPTH is **not allowed at all on a print panel** — there the areas simply flow onto the page.

- **MAXWIDTH sizes the column, not the data** — it must cover the *wider* of the value and the column heading. A CHAR(10) name under a heading reading **Environment** needs MAXWIDTH=11, because the heading is the wider of the two. Sizing it from the data alone is the easy mistake, and it is caught by CRTPNLGRP rather than at run time, so it costs a compile rather than a defect.
- **Each WORD of a column heading goes on its own line — apostrophes are what hold them together.** The manual is explicit, in the same wording for :LISTCOL and :DATACOL:

"Each word of the column heading is placed on a new line. If multiple words are necessary in a line of the column heading, they must be enclosed in apostrophes ('). Each word or quoted string must fit within the maximum column width defined by the MAXWIDTH attribute of this tag. The maximum number of words or quoted strings allowed is specified by the MAXHEAD attribute on the LIST tag."

So `.Merge Env` draws on two lines and `. 'Merge Env'` draws on one. **The multi-line form is a default, not a constraint** — which is worth stating because the default is what you get by accident, and it reads like the tag refusing to do what you asked.

Three consequences, and the second is the one that bites:

- **MAXHEAD counts words or quoted strings**, not lines of text.
- **The limit a quoted heading must fit is MAXWIDTH, not the layout column.** A heading quoted into one line still has to fit the column it heads, so widening LAYOUT does not buy room for it.
- Column data is **centred** under a wider heading, the slack going right for JUSTIFY=LEFT.

`'Merge Env'` is 9 against MAXWIDTH=10, so it fits and stays on one line. `From Environment` in `WRKENVXF` is 16 against MAXWIDTH=11, so it cannot be quoted onto one line at all and MAXHEAD=2 is genuinely required — the two cases look alike and are decided by that comparison.

Prefer a heading that fits on one line, and abbreviate to get there — `Merge Env` over `Merge Environment`. A two-line heading costs a list line, which at LAYOUT=6 is six entries a page.

- **Size every list column, and oversize the last one on purpose.** UIM maintains a separator of *"two to five characters"* between list columns and chooses the width itself, spending whatever is left after the columns are placed. On a list of narrow columns that means wide gaps and a row strung right across the panel.

The lever is the arithmetic, not an attribute. Give the last column more width than the row can hold and there is no slack to spend: every separator falls to its two-character minimum and that column truncates at the right margin. `WRKOBJTM` and `WRKTSKTM` are the worked examples — text columns at 48, 50, 92 and 100 characters on an eighty-column list, `WRKTSKTM`'s first view asking for 121 characters of column before separators.

- **MAXWIDTH='*' does the opposite of what it reads like.** *"If '*' is coded, the remainder of the area width is used for the column"* — but the remainder is what is left **after** UIM has placed the separators, so the '*' column hands UIM the slack rather than denying it. The gaps go wide and the column absorbs the loss. No panel in these repos uses it.

It has a second cost that only shows up later: a '*' column may not sit inside a :LISTGRP, and the compiler says so at the point you reach for a group to fix the spacing '*' caused.

- **:LISTGRP is not the answer to column spacing, and is rarely the answer to anything here.** It carries COLSEP=N, which is the only *attribute* that sets a separator width, so it looks like the fix — but it is the expensive route to what the paragraph above gets for free, and it is worth knowing why before reaching for it:
 - **Grouping moves the help up to the group; it does not add a level.** HELP= is required on :LISTGRP and **forbidden** on a :LISTCOL inside one: *"The HELP attribute is not allowed if the column is part of a list column group defined by the LISTGRP tag, but is required if the column is not part of a group."* So four columns pulled into a group to fix their spacing lose their four help modules and share one.
 - **Neither the action column nor a MAXWIDTH='*' column may be in a group**, so Opt and the text column stay outside it either way.
 - **COLSEP governs only the gaps inside the group.** *"This attribute does not affect the space between list column groups and other list columns."* The gaps on either side are still UIM's to choose.

The compiler charges the help rules in two instalments — first the missing HELP on the group, then HELP attribute on LISTCOL tag already specified on LISTGRP tag on the first column — so a fix that answers only the first message is not finished.

Use a group when the columns genuinely belong under one heading and one help panel. Do not use one to control spacing.

- **A function key whose legend changes is two :KEYI tags for the one key, each with a complementary COND=.** This is the only way to do it, and the obvious way does not exist: the manual is explicit that key-description text *"must appear on the same or next line as the tag and can only contain the reverse text (RT) tag"* — so no &variable. substitution, and the legend cannot be built by the program.

```
:KEYI    KEY=f14
          COND=notexcl
          PRIORITY=1
          HELP='WRKXRFXF/HLPF14'
          ACTION='return 14'
          VARUPD=no
          .F14=Exclude duplicates
```

```
:KEYI    KEY=f14
          COND=isexcl
          ...
          .F14=Include duplicates
```

with a dialog variable the program writes on the panel's header record, and two conditions over it:

```
:COND      NAME=notexcl  
           EXPR='hexcl *EQ "0"'.  
:COND      NAME=isexcl  
           EXPR='hexcl *EQ "1"'.  

```

Naming the same key twice in one :KEYL is accepted — COND= says the key "*is in effect on the panel only if the condition specified is true*", and complementary conditions mean exactly one tag is ever in effect, so exactly one legend is drawn. The manual does not say in so many words that a key may appear twice; WRKXRFXF compiles and runs this way, which is the evidence for it.

Word the legend as what pressing it WILL do, not as what the state is.

F14=Exclude *duplicates* on an expanded list and F14=Include *duplicates* on a collapsed one. A legend naming the current state reads as a label rather than an action, and leaves the user to work out which way the key goes.

The state variable is a dialog variable like any other, so it belongs in a :VARRCD and in the matching RPG structure — even though nothing displays it. Say so in a comment at both ends; a field in a header record that never appears on the panel looks like a leftover.

7. Application display programming (UIM)

Section 6 covers panel *source*. This covers the *programs* — the driver that opens the application and loads the list, and the exit program UIM calls back.

Most of what follows is not discoverable from the panel source, and is only half-discoverable from the API reference. It was established the expensive way, by building WRKENVXF, and it is written down here so the next work-with panel does not pay for it again. Where a rule below looks arbitrary, it is usually the record of a failure that took a day to identify.

Read the manual before reasoning from first principles. *Application Display Programming*, SC41-5715, is the panel group tag reference — every tag, every attribute, with the formatting rules that are otherwise discovered by compile error. It answers most of what this section had to work out the hard way, including several things assumed impossible here before it was consulted. It is in the local manual directory named at the top of this document.

7.1 Two programs, no compile-time link

A work-with panel is a **driver** and an **exit program**:

	Responsibility
Driver (xxxWRK)	Opens the display application, builds the list, displays the panel in a loop, handles the function keys that return nn
Exit program (xxxEXT)	Everything UIM calls back for — list options, panel checking, post-action

Nothing binds a program to its panel group at compile time. Unlike a display file, a panel group is resolved *by name at run time*: the panel id, the :VARRCD name and the :LISTDEF name are character strings the program hands to an API. Get one wrong and the compiler is silent — the failure is a UIM escape at the moment that path is first exercised, which may be days later.

That is the whole reason for the @..._PNLID / @..._VARRCD / @..._LSTNAM constants of section 1.5. They are not tidiness; they are the only mechanism keeping the two members in step, so **every** name crossing the boundary is a named constant and never a literal.

7.2 Call types

The exit program's first act is to branch on `typCall`. **Name the call types** — @QUIM_TYPCALL_* in QUIMH — rather than testing bare numbers.

Declared on	Call type	Fires
:KEYI ACTION='CALL EXITPG'	1 FNCKEY	when that function key is pressed
:LISTACT ENTER=/ PROMPT='CALL EXITPG'	3 LSTOPT	once per selected entry, to <i>perform</i> the action
:PANEL USREXIT='CALL EXITPG'	4 PNLCHK	once for the panel, at input validation time
:LISTACT USREXIT='CALL EXITPG'	5 LSTEXT	once per selected entry, after the action

Two of these are routinely confused, and the difference decides the design:

- **USREXIT on a :LISTACT is not USREXIT on a :PANEL.** The first is a per-entry post-action (type 5), the second a whole-panel exit (type 4). They share a keyword and nothing else.
- **Type 4 is validation, not post-processing.** It runs while the panel's input is being checked, which is *before* any list action on that panel — so it is the place to reject input, and never the place to react to work already done. Reaching for it as an "after everything" hook is the mistake it invites.

7.3 A list action can run the command itself

An exit program is **not required** to perform an option. A list action may name a command, and UIM runs it per selected entry:

```
:LISTACT ENTER='CMD CPYENVXF FROMENV(&ENV.) TOENV(&TOENV.) '  
        PROMPT='CMD ?CPYENVXF ?*FROMENV(&ENV.) TOENV() '  
        USREXIT='CALL EXITPG'
```

Prefer this to building the same command string in RPG. It is declarative, the prompted and unprompted forms sit together where they can be compared, and the exit program is left with only the post-action.

The variables resolve against the ACTION entry — the entry of the list the :LISTACT belongs to — not against whatever panel is on the screen when the user types. Section 7.5 is a direct consequence of that sentence.

7.4 :VARRCD and :LISTDEF are different things

Both list variable names, and they are not interchangeable:

Tag	What it is
:VARRCD	An I/O buffer layout . Named on every get/put/add/update call, and a byte-for-byte match of the RPG data structure passed with it.
:LISTDEF	The set of per-entry variables a list carries, plus its EMPHASIS slots. Not a buffer.

Neither is a subset of the other. A variable may sit in a :VARRCD and not in the :LISTDEF, and — the case that matters — **in a :LISTDEF and in no :VARRCD at all**, which is exactly what a confirmation list's variables do.

Two failure modes follow, both worth recognizing on sight:

- **"Variable buffer length too small"** means the record the panel declares is longer than the buffer the program passed. The usual cause is not a coding error but a **stale object**: a field was added to the copybook and only some of the programs that /copy it were rebuilt. Recompile every consumer, not only the member that changed.
- **A varying field in a dialog variable record is silent corruption.** A Varchar carries a 2-byte length prefix, which shifts every field after it and renders the text as garbage. Type these Char(n) even where the column is VARCHAR(n) and section 2.4 would otherwise call for Like() — and say so in a comment, or the next reader will "fix" it back.

7.5 Confirmation lists belong to UIM

CONFIRM= on a :LISTACT names a panel that shows the selected entries. **UIM builds and owns that list.** No program adds to it: its :LISTDEF names a **subset of the action list's variables, matched by name**, and UIM copies the selected entries into it.

Three consequences:

- **A column the user types into on a confirmation panel must also be a variable of the action list.** Declared only on the confirmation :LISTDEF, it has no per-entry storage on the entry the action runs against, and the substitution in section 7.3 resolves to blanks. Carry it on the main :LISTDEF even though no :LISTCOL on the main panel displays it.
- **UIM does not carry the typed value back to the action entry.** Nothing does. Give the confirmation panel a :PANEL USREXIT, read its list in the type 4 exit, and write the value onto the matching action entry. By the time UIM resolves the command, the value is there.
- **The confirmation list is addressable only while its panel is current.** Reading it from an exit called on the *main* panel fails with CPF6A92, *List &4 not active* — a statement about scope, not about the approach.

This is the mechanism we arrived at, not one we found. The behaviour is familiar from PDM, whose 7=Rename shows the selected members with an input column for the new name and processes them all on one Enter — but PDM is not a UIM application (section 7.9), so it is not evidence about what UIM does. No UIM example of an input column on a confirmation panel has been located, in these repos or in the documentation.

Treat this section as the route that works. **Do not read the amount of machinery as proof that a shorter path exists** — the screen that suggested one turned out to be a different technology, so there is nothing left implying UIM has a neater answer. This may simply be what UIM costs. If a genuine UIM example turns up, revisit before copying this again.

7.6 Reading and positioning list entries

QUIGETLE takes a **position option** saying which entry to return:

Option	Returns
FRST / NEXT / PREV / LAST	a plain walk
HNDL	the entry named by a handle — what an exit program uses to read the entry it was called for
	the first/next entry satisfying a selection criteria

Option	Returns
FSLT / NSLT / PSLT / LSLT	

The *SLT options require the Selection Criteria parameter, and it is not optional — omitting it is CPF6A2D, *Value for Selection Criteria parameter not valid*. The criteria is a relational operator plus **the name of a dialog variable**; UIM compares that variable's current value against the value held in each list entry.

So a single entry is **fetches, not searched for**:

```
sltCri.relOpr      = @QUIM_RELOPR_EQ;
sltCri.dlgVarNam = @XFENVWRK_VAR_ENV;
```

with @QUIM_LSTPOS_FIRST_SLT. Set the dialog variable first — reading another entry with copy-to-dialog-variables on is one way, an explicit QUIPUTV the other — and **the two calls are then ordered, not merely adjacent**. Say so in a comment: nothing in the source makes that dependency visible, and separating the calls breaks the second one silently.

Prefer this to walking. Walking the list once per selected entry is a full scan per action — invisible on a ten-row list, embarrassing on a long one.

7.7 The list is maintained, not reloaded

Every list change is an entry-level operation, performed by the exit program on the callback for that entry:

Option	Post-action does
Change	QUIUPDLE the entry
Remove	QUIRMVLE the entry
Copy / create	QUIADDLE the new entry, positioned to keep the list's order

Do not look for an "after all list actions" hook. There is none. UIM processes the selected entries and redisplay the panel internally; the driver does not regain control, and the panel's ENTER= action fires only when nothing is pending. The observable symptom is that work done by the options is not on the panel until the user presses F5.

A **full reload belongs to the driver**, reached the only way the driver can be reached: a function key whose action is `return nn`, after which the driver runs its command and calls its load routine.

F6=Add sits in the driver for that reason — but the reason does not hold. Six members across the WRK*TM family carry the same note, in each driver and its exit program:

```
// HOWEVER, the panel will use either one or the other
// based on the action for the F6 key being set to
// 'return 6' (handled by TMLOCWRK) or 'CALLE EXITPG'
// (handled by TMLOCEXT). The problem of using the
// exit handler program to add the location is the ability
// to refresh the location list ... not sure how to do that
// without re-coding the entire load list routine.
```

Read it for what it is: a **function key** whose work happens to create a row, and an exit program with no access to the driver's load routine. It says nothing about list actions, and it is not evidence that a hook is missing.

Its premise is also wrong, which is the more useful lesson. A new row does not need the list rebuilt — it needs one entry inserted, which is QUIADDLE and is exactly what the copy post-action in the table above does. The exit program can service F6 perfectly well. The note reads as a dead end only because it equates *refresh* with *reload*, and once a list is understood as maintained per entry rather than reloaded, the difficulty disappears.

Watch for that equation in your own reasoning; it is what sends people looking for a hook instead of writing three API calls.

Insert a newly created entry whatever the active subset says. The user has just created the thing, and a list that does not show it reads as a command that failed. It drops out at the next genuine reload, which is the right moment for a filter to be applied — not the moment of creation.

Reloading from a post-action would rebuild the whole list once per selected entry and throw away the list position each time.

7.8 A value written to a list entry stays there

A list entry is storage, not a screen field. A value written onto it for one action is still there for the next, so a second use of the option arrives prefilled — which no other option does, and which is a defect rather than a convenience.

Clear it in the post-action, along with the option number, exactly as a change post-action clears the option. Clear it on the **success** path only: where the command failed, what the user typed is what they need in order to correct it.

7.9 PDM is not a UIM application, and is not a model for one

PDM's screens come from a **display file** — QPDA/QDUODSPF. UIM panels are not display files of their own; they are routed through QDUI80 and QDUI132. So every work-with behaviour PDM exhibits is DDS subfile behaviour, and none of it is evidence about what UIM can be made to do.

This matters because PDM is the work-with panel everybody knows, and it is the obvious thing to reach for when deciding how a new one should behave. Time has been lost here trying to reproduce PDM behaviour *exactly* in UIM, on the assumption that IBM had done it with the same tool and the technique was simply undiscovered.

The option field is the tell, and it is visible without checking any object:

	PDM	UIM
Option value	alphanumeric	numeric
Available options	extensible at run time through a user options file	every one declared in the panel source at compile time

A screen accepting user-defined two-character options is not a UIM list. Where a UIM panel needs an option UIM does not offer, it has to be coded — there is no options file to extend.

Check before treating any IBM screen as precedent. `WRKOBJ OBJ(<lib> /*ALL) OBJTYPE(*PNLGRP)` answers it: a UIM application has panel groups, and `RTVPNLGRPSRC` will then give you the source to read. An application with none is not one to copy from.

8. Display files (QDDSSRC)

- Standard header applies: `A*` comment lines carrying the purpose, `MODIFICATIONS:` log, and `*>` build directives (`CRTDSPF`, continued with `-`).
- **Record formats are named FMnn?** — `FM`, a two-digit number `nn` (01–99), and a type suffix. Formats that work together (a subfile and its control, a window and the records displayed in it) share the same `nn`:

Suffix	Format
<i>(none)</i>	Plain record
S	Subfile record (SFL)
C	Subfile control (SFLCTL)
F	Footer
B	Blank
W	Window

- Note: these repos hold only a handful of display files from a much larger code base, so local DSPF members are weak precedent. Follow the naming rule as stated here rather than patterns inferred from the few local examples.
-

9. Messages

8.1 Message identifiers

A message id is a **3-character prefix and a 4-character number**. Compose the prefix as a **2-character module id plus a type letter**, following IBM's own scheme in QCPFMMSG:

Letter	Type	IBM's
A	Action — inquiry messages expecting a reply	CPA
C	Completion	CPC
D	Diagnostic	CPD
I	Informational — including *STATUS	CPI
X	Text — titles, help text, NLS and cultural values	CPX
F	Everything else — escape, notify	CPF

So the Cross Reference module sends XFF*, XFC*, XFD* and XFI*. (IBM's F stands for *facility*; CPF as a whole is System/38 compatibility. The letter is worth keeping anyway, because a module needs a bucket for the messages that are not one of the named types.)

Match the letter to the send type: *COMP is C, *DIAG is D, and both *INFO and *STATUS are I — a status message is informational, not an "everything else".

But the letter records what the message IS, not every way it is ever sent — and the test is where it could have ORIGINATED. *DIAG is the one send type that may be an escape in transit: forwarding an escape as a diagnostic is routine, and is exactly what the FWDMSGH pattern of section 3.4 does. So a message sent *ESCAPE in one place and *DIAG in another stays an F message being used as a diagnostic.

That is also why *STATUS is unambiguous where *DIAG is not. A *STATUS message is never sent as *ESCAPE and never arrives by being forwarded from one, so it has no other origin to weigh — it is I and nothing else.

Number by area within the prefix, so related messages read together: XFF0nnn for the build, XFF1nnn for environment definition. The number is hexadecimal — A–F are legal digits — but **do not use them in user-defined ids**. Reserve that for a custom version of an IBM message whose id already contains them.

8.2 One module id per module

Every message a module owns carries the same 2-character module id. Only the type letter varies.

This exists because messages are monitored by prefix group. MONMSG

MSGID(CPF0000) catches every CPF message; a module that spreads its messages across unrelated prefixes forces every caller to name each one:

```
MONMSG      MSGID( ... TSK0000 OBJ0000 LOC0000 DEP0000 +  
              EXC0000 UIM0000) EXEC(GOTO CMDLBL(STDERR))
```

Six prefixes for one module's failures, and a caller that names five of them misses the sixth **silently** — the escape is simply not caught, and the failure surfaces somewhere else as something else.

Splitting by type letter does not reintroduce the problem, which is the point of putting the type in the prefix rather than in the number range. Only escapes are monitored, and escapes are all F, so a single xxF0000 covers what a MONMSG needs. Completion, diagnostic and informational messages are read, not trapped.

Task Manager is the standing exception, not a model. TMMSG predates this rule and uses TSK*, OBJ*, LOC*, DEP*, EXC* and UIM*. It is left alone because the remapping is genuinely hard — the areas hold colliding numbers, so they cannot simply be re-prefixed — and it is tracked in GCUTL/QRPGLESRC/READMETM.MD rather than done piecemeal. New modules follow the rule.

8.3 Sending

- Product messages live in one message file per product (XFMSG, TMMSG), built from a QREXSRC REXX member.
- Programs send message IDs, not hardcoded text, wherever a message exists; status messages use CPDA0FF.
- **Keep a *STATUS message to 76 characters assembled.** That is the text plus the *declared width* of every replacement value, not the width of what a particular caller happens to pass. Longer messages are truncated, and possibly not shown at all — either way the tail is lost, and the tail is usually the part that says what the program is doing.

Measure it when the message is written: literal text with the &n tokens removed, plus each value's FMT length. 'Building cross reference &1. &2' with (*CHAR 10) (*CHAR 30) is 28 + 10 + 30 = 68.

- **MSGDTA must supply exactly the bytes the FMT declares.** A short MSGDTA leaves later replacement values reading whatever follows in storage, which shows as stray

characters rather than as an error. Concatenating a 10-byte variable with a shorter expression is the easy way to get this wrong:

```
CHGVAR      VAR(&STSTXT) VALUE('Scanning' *BCAT &LIB)
SNDPGMMSG   MSGID(XFI0007) MSGF(XFMSG) MSGDTA(&ENV *CAT +
          &STSTXT) TOPGMQ(*EXT) MSGTYPE(*STATUS)
```

&STSTXT is declared at the FMT width, so the total is right whatever the text inside it happens to be.

- **No apostrophe in message text.** A QREXSRC message-file builder hands MSG and SECLVL to ADDMSGD inside CL quotes, so an apostrophe in the text closes the CL string early and the command fails with

CPF0001 Error found on ADDMSGD command.

which names the command and says nothing about quoting. It surfaces on the *one* message that has one, so a file of forty builds thirty-nine and stops.

Doubling it for CL does work, and costs **four** apostrophes in a REXX literal — `job' ' ' 's` yields `job' 's`, which CL renders as `job' s`. Reword instead: "the job library list", not "the job's library list". The possessive earns nothing in message text and the quadruple apostrophe is a trap for whoever edits the line next.

- Escapes propagate (FWDMSGH pattern / `%target('*PGMBDY' : 1)`) so the original failure reaches the user.

10. Cross-cutting rules

- **Self-documenting builds:** if it can't be rebuilt from its BLDOBJ directives, it isn't done.
- **Re-runnable everything:** DDL re-creates, programs clean up QTEMP, handled messages are removed from the joblog.
- **No duplicate knowledge:** lengths via `like()`, help via `IMHELP`, message text via `&msg( ... )`, schema via DFTRDBCOL resolution. One source of truth each.
- **Order dependencies are documented at both ends** — e.g. the build-order CASE in RUNTSKTM and TMOBJWRK's sort carry matching "keep in sync" comments. Do the same for any new coupled logic.
- **Coded values are numeric, not mnemonic.** This governs *codes* — values where a character stands in for a word the reader has to know: status, option, scope, selector. It has nothing to say about values that are already words, such as IBM-style special values (`*LIBL`, `*MERGE`) or text labels (`STEP07`); those are names and stay as they are. Codes store digits. A single letter is ambiguous the moment someone other than its author reads it: C is Continue, Confirmed, Cancel, ... A digit

makes no false promise, so the reader goes to the column text or the panel for the meaning instead of guessing wrong.

- Number from 1 in the natural progression of the thing, and reserve **9 for the error or terminal state**, leaving room for states added later.
 - Where the values are nested scopes rather than distinct states, number them in widening order so the ordering itself carries meaning.
 - Keep the column `CHAR(1)` holding a digit — it is a coded value, not arithmetic — and spell every value out in `LABEL ON COLUMN ... TEXT IS` so the decode travels with the table.
 - Write that legend as `value=Description`, description in **sentence case**, values separated by **comma + space**: `Status: 1=Running, 2=Complete, 9=Error`. Column `TEXT` caps at **50 characters** — abbreviate the descriptions to fit and keep the authoritative decode in the design document; do not let the legend run past the cap and truncate mid-word, which is how IBM's own `QADSPPGM` legends ended up unreadable.
 - No `UPPER()` normalization for these columns in the `<table>T1` trigger; it is dead code once the domain is numeric.
 - The readable form belongs at the **command layer**, spelled out in full: `SPCVAL((*STDIP 1) (*DEVIP 2) (*ALL 3))`. The user types the special value, never the digit.
-

11. Legacy and scratch code

- **Scratch:** `JUNK*` members are experiments. Git-ignores them going forward; never reference or promote them, and don't cite them as precedent.
- **Third-party:** members with external authorship headers — Carsten Flensburg's `CPYMSGD*`, Giuseppe Costagliola's `SQL2XLSX*`, Scott Klement's `YAJL/JDBCR4` material — keep their original style and attribution. Fix bugs; don't restyle.

The reason is practical, not ceremonial: their style is the upstream author's, so restyling makes the next version harder to take up and makes a local defect harder to tell from an upstream one. **Where that reason has expired, so has the exemption.**

- **Forks that have diverged are house code with an attribution.** A member branched from someone else's work and then rewritten over years is no longer tracking an upstream. Nobody is going to merge a new release into it, so nothing is being protected by holding it to the original author's conventions — the exemption just leaves a permanent inconsistency in the middle of the library.

`BLDOBJ` is the worked example. It began as a branch of Klement's `BUILD` tool in 2019 and has drifted a long way since. **The @author line stays** — the origin is a fact and the credit is owed — but house standards apply to the code. The test is

not "was this written elsewhere" but "is there still an upstream we intend to follow." Keep the attribution, drop the exemption.

- **Legacy fixed-form:** don't expand fixed-form code. Small fix = match the existing style locally; real enhancement = convert to ****FREE** first (add a **MODIFICATIONS** entry for the conversion).
- **Embedded 5250 display attributes become a single space.** Old source can carry EBCDIC **x'20'–x'3F'** — the 5250 field attribute bytes. They are there because the comment or heading was laid out on a green screen, where an attribute byte turned highlighting or colour on and off. Converted off the platform they surface as Unicode C1 controls, mostly **U+0080–U+009F**.

Replace each with exactly one space. An attribute byte occupies one display position on a 5250 — it is not zero-width — so one space is what preserves the column alignment the original author was looking at. That is the whole justification, and it is why the two obvious alternatives are both wrong:

- **Leaving them** means the next tool to read the member decides for you. One that cannot decode the byte writes **U+FFFD** in its place, which is silent corruption that survives review because it still looks like a character.
- **Deleting them** closes up the line. **NOTE:** + attribute + **WHEN** becomes **NOTE:WHEN**, and in CL it can be worse than cosmetic — an attribute sitting before a + continuation is holding the blank that separates it from the preceding keyword, so removing it changes what the compiler reads.

Byte-count matters as much as appearance here: source is fixed-width (see §1.4), so a substitution that is not one-for-one shifts everything after it.

They do not all land in the C1 range — match on the whole set. EBCDIC **x'20'–x'3F'** converts to a scatter of Unicode code points, not a contiguous block: most become C1 controls (**U+0080–U+009F**), but **x'26'** becomes **U+0017**, **x'2F'** becomes **U+0007**, **x'3F'** becomes **U+001A**, and several more land in C0. A pattern written against **U+0080–U+009F** alone looks like it works and quietly leaves the C0 ones behind. Match **C0 except tab/LF/CR, all of C1, and U+FFFD**:

```
[\\u0000-\\u0008\\u000B\\u000C\\u000E-\\u001F\\u007F\\u0080-\\u009F\\uFFFD]
```

x'25' is the one to leave alone — it converts to **U+000A**, a real line ending.

Do not widen this to "any non-ASCII". A handful of members genuinely contain extended characters — a few across a thousand-plus — and those are content, not artefacts. The rule is scoped to control code points precisely so a sweep cannot eat them.

- **Write new source in ASCII, and treat anything outside it as needing a reason.** Source reaches the system through the IFS and is translated to EBCDIC on the way in. A character with no equivalent in the target CCSID is replaced with the

substitution character, **x'3F'** — silently, with no warning and no way to recover what it was.

Most non-ASCII in source is a *choice* rather than a requirement, and the choice is free to make differently: an em-dash where a hyphen reads the same, >> where >> does. Use the ASCII form. **The test is whether an ASCII alternative conveys the same thing, not whether one exists.**

Occasionally it does not — a visual marker or a simulated bullet that no ASCII character stands in for. Those are content and stay, as above. Verify one survives the round trip on the system rather than assuming it will.

x'3F' hides inside the display-attribute sweep. It falls within x'20'–x'3F', so the rule above replaces it with a space along with the genuine attribute bytes — erasing the evidence that a character was lost in translation, and leaving the two indistinguishable afterwards. Check a member for substitution characters *before* sweeping attributes, not after.

This is worth a check rather than a habit, since nothing surfaces it: a byte scan for anything above 127 across the source directories catches it before the member ever reaches the system.

- **Control codes in DATA you are about to display: replace the whole x'00'–x'3E' range with x'3F', mark the positions, and restore after the read.** This is the same byte range and the same substitution character as above, in the other direction — not characters that arrived in source, but characters a program is about to send to a 5250.

The failure is worse than a bad character. Writing a control code to the display can end the program in error, and has been seen to drop the session outright. So this is not a cosmetic rule; it is what keeps a display program alive when it meets data it did not expect.

The data does not have to be text for this to happen. The case that produced this was a display of an Island Pacific formatted LDA, whose program option areas are *supposed* to hold character and signed numeric data and overwhelmingly do. The fraction of procedures putting packed or binary values there is small — a couple of percent — and depending on the value, those bytes land in the control range. A display that works on almost everything is one unusual caller away from killing a session.

x'3F' is the replacement because it round-trips. It reaches the display and comes back to the program unchanged, which is what makes the substitution reversible. Marking each replaced position and restoring the original after reading the screen is what keeps it lossless — without that step the substitution corrupts the data it was protecting.

Replace the range, do not filter it. The obvious instinct is to substitute only the codes that actually misbehave, and it has been measured: too few of x'00'–x'3E' can be displayed safely for selectivity to be worth the logic. Take the whole range. The experiment is not worth running again.

- **The standard applies to each repository on its own merit.** It travels to every Godsend IBM i repo, but says nothing about the relationship *between* them. Nothing here requires two repos to hold identical members, and divergence between them is not a standards violation — a developer library drifts from a curated one as a matter of course, work gets tested and abandoned, and other hands change production. Where a change is made because of a standard and belongs in more than one repo, **apply it in each repo separately, judged against that repo's own code.** Never reconcile one repo by copying from another on the strength of this document.

What *is* kept identical is this document. Edit the canonical copy in **IBMI-STANDARDS**, run **sync-standards.ps1**, and commit the refreshed copy everywhere it lands. How a particular pair of repositories relate — which is authoritative for what, what is being migrated where — is that project's business and belongs in that project's own documentation.